같은 규칙을 쓰는 다른 실험도 다시 돌렸다. 적응형 BS23의 결정 열거(20.7절)는 네 대상 모두 경로 수까지 같은 판정이 나왔다. 결정 열거를 쓰는 분야(신경망 추론, 잔액 이전)의 원본·변이체 221개도 판정이 하나도 바뀌지 않았다. 잔액 이전은 입력이 사전에 들어 있어 처음에는 변수로 읽히지 않았고, 이를 고친 뒤의 결과이다.

23 제5부 코드

20–22절의 판정은 모두 아래 코드로 다시 낼 수 있다. 1–19절의 코드(제4부)는 그대로 쓰고, 고친 곳은 없다.

oracle_close.py — 정확 산술 재실행(S·G), 정수 추적과 대표 크기(T), 결정 열거(D), 나무 ODE 차수(B)

```
"""oracle_close: 수치해석의 오라클 문제를 닫는 층 — 실행한 입력에서 모든 입력으로(20절).

검사 순서의 앞 셋(공간·시간·정확도, 1–19절)은 실행한 계산에서 식과 값의 어긋남을 빠짐없이 잡는다.  이 모듈은
남은 한 층, 곧 "실행하지 않은 입력"을 닫는다.  코드를 부동소수 대신 두 가지 정확한 산술로 실행해, 코드가 계산하는
식 자체를 의도한 식과 비교한다.

  (S) 기호 실행     상태·걸음·격자를 sympy 기호로, 우변 f를 해석하지 않은 함수로 두고 코드를 그대로 돌린다.
                    코드의 출력은 (유리 연산 + 해석하지 않은 함수 적용)의 식이고, 정규형(함수 인자까지 cancel)이
                    의도한 식과 같으면 모든 f·상태·걸음·격자에서 같다(결정적 증명).  한 걸음·한 격자 크기에 쓴다.
  (G) 유한체 실행   GF(p), p = 2^61 - 1 위의 무작위 값과, f 자리에 무작위 신탁(같은 인자에는 같은 값, 다른 인자에는
                    독립 균등값)을 넣어 코드를 돌리고, 걸음마다·마디마다 의도한 관계를 정확히 비교한다.
                    식이 다르면 다른 값이 나올 확률은 1 - deg/p 이상이다(슈바르츠–지펠, 해석하지 않은 함수는
                    아커만 환원).  비교 하나의 오판 확률 상한을 증명서에 적는다.
  (T) 정수 추적     걸음 수 n, 격자 크기 N, 그리고 그것에서 나온 루프 번호를 추적 정수로 넘긴다(range, len, enumerate를
                    대상 모듈 안에서 바꿔 끼운다).  정수에 걸린 술어(상수와의 비교, 나머지, 정수 나눗셈)를 모두
                    기록하고, 술어의 참거짓 칸마다 대표 입력을 골라 (G)로 판정한다.  칸 안에서는 제어 흐름이 같으므로
                    대표들이 통과하면 모든 n, N에서 통과한다.
  (B) 나무 ODE      주장도 목록도 없을 때: 뿌리 나무 t마다 꼭짓점이 변수인 다항 ODE  y_v' = prod_{c 자식} y_c
                    (y(0) = 0)를 만들면, 정확해의 뿌리 성분은 h^|t| / gamma(t)이고 한 걸음 방법의 뿌리 성분에서
                    h^|t|의 계수는 그 방법의 기본 무게 b^T Phi(t)이다.  코드를 유리수·기호 h로 한 걸음 돌려
                    계수를 정확히 읽으므로, 계수표를 몰라도 코드 자체의 차수를 정확히 판정한다.

판정은 증명·반례·정의 가운데 하나이다.
  반례     (G) 또는 (S)에서 의도한 식과 다른 값.  (n 또는 N, 걸음 또는 마디, 기대값과 다른 값) — 언제·어디서.
  증명     (S) 정규형 일치(결정적), 또는 (T)의 모든 칸에서 (G) 일치(오판 확률 <= 적힌 상한).
  범위 밖  (정의) 값에 따라 갈리는 분기(GF에는 순서가 없으므로 비교가 일어나면 즉시 멈춘다), 추적 밖의 정수 경로
           (전역 변수, numpy shape, 시계 등)로 들어온 분기.  이것은 판정 범위의 정의이고 보류가 아니다.
"""
import ast, builtins, inspect, itertools, math, random, textwrap, types
from fractions import Fraction as Fr
from functools import reduce
import numpy as np
import sympy as sp

P = (1 << 61) - 1                       # 메르센 소수
U = 2.0 ** -53


# ---------------------------------------------------------------- 부동소수 상수 -> 그 상수가 반올림한 유리수
def rat(c):
    """부동소수 c가 반올림한 유리수 p/q (|c - p/q| <= 4u|c|, q <= 2^24, 연분수의 첫 수렴값) — 없으면 c 그대로의 유리수.
```

```
    코드에 적힌 1/6, 0.5 같은 상수는 이렇게 '적힌 식'의 유리수로 읽는다."""
    if isinstance(c, (int, Fr)): return Fr(c)
    x = Fr(float(c))
    if x == 0: return x
    a, b = x.numerator, x.denominator; h0, h1, k0, k1 = 0, 1, 1, 0
    while b:
        q = a // b; a, b = b, a - q * b
        h0, h1 = h1, q * h1 + h0; k0, k1 = k1, q * k1 + k0
        if k1 > 1 << 24: break
        if abs(Fr(h1, k1) - x) <= 4 * U * abs(x): return Fr(h1, k1)
    return x


class ValueBranch(Exception):
    """GF 값에 대한 비교: 값에 따라 갈리는 분기(판정 범위 밖, 정의)."""


# ---------------------------------------------------------------- GF(p)
PIN = [True]            # 같음 갈래의 경로 조건 대입(22.10절).  False면 고치기 전 규칙: 같음 갈래를 무작위 점에서 판정
_LIN = [False]          # 결정 열거 중에만 켠다: GF 값마다 입력 변수에 대한 1차식(아니면 None)을 함께 나른다


def _vars(l): return {x for x in l if x is not None and x != "~"} | set(l.get("~", ()))
def _pure(l): return all(x is None for x in l)


def _lin(a, b, k):
    """값의 변수 구조.  식은 {변수: 계수, None: 상수, "~": 1차가 아닌 곳에 나온 변수의 집합}.
    계수 자리의 변수는 그 값에 1차(계수는 상수)로만 나온 변수이다.  모르는 값(None)이 끼면 None."""
    if a is None or b is None: return None
    if k == "+" or k == "-":
        r = dict(a); sg = 1 if k == "+" else -1; nl = set(a.get("~", ())) | set(b.get("~", ()))
        r.pop("~", None)
        for x, c in b.items():
            if x == "~": continue
            v = (r.get(x, 0) + sg * c) % P
            if v: r[x] = v
            else: r.pop(x, None)
        if nl:
            for x in nl: r.pop(x, None)
            r["~"] = frozenset(nl)
        return r
    if k == "*":
        if _pure(a): c, f = a.get(None, 0), b
        elif _pure(b): c, f = b.get(None, 0), a
        else: return {"~": frozenset(_vars(a) | _vars(b))}
        r = {x: v * c % P for x, v in f.items() if x != "~" and v * c % P}
        if "~" in f and c: r["~"] = f["~"]
        return r
    if k == "/":
        if _pure(b) and b.get(None, 0):
            inv = pow(b[None], P - 2, P); r = {x: v * inv % P for x, v in a.items() if x != "~"}
            if "~" in a: r["~"] = a["~"]
            return r
        return {"~": frozenset(_vars(a) | _vars(b))}
    return None


class GF:
    __slots__ = ("v", "l")
    def __init__(s, v, l=None): s.v = v % P; s.l = l
    @staticmethod
    def c(o):
        if isinstance(o, GF): return o
        if isinstance(o, Opq): return o.as_gf()
        if isinstance(o, bool): raise TypeError("bool")
        if isinstance(o, (int, np.integer)): v = int(o) % P; return GF(v, ({None: v} if v else {}) if _LIN[0] else None)
        if isinstance(o, (float, np.floating, Fr)):
            r = rat(o); v = r.numerator * pow(r.denominator, P - 2, P) % P
            return GF(v, ({None: v} if v else {}) if _LIN[0] else None)
        return NotImplemented
    def _b(s, o, f, k, rev=False):
        o = GF.c(o)
        if o is NotImplemented: return NotImplemented
        r = f(o)
        if _LIN[0]: r.l = _lin(o.l, s.l, k) if rev else _lin(s.l, o.l, k)
        return r
```

```
    def __add__(s, o): return s._b(o, lambda o: GF(s.v + o.v), "+")
    __radd__ = __add__
    def __sub__(s, o): return s._b(o, lambda o: GF(s.v - o.v), "-")
    def __rsub__(s, o): return s._b(o, lambda o: GF(o.v - s.v), "-", True)
    def __mul__(s, o): return s._b(o, lambda o: GF(s.v * o.v), "*")
    __rmul__ = __mul__
    def _inv(s):
        if s.v == 0: raise ZeroDivisionError("GF: 0으로 나눔")
        return pow(s.v, P - 2, P)
    def __truediv__(s, o): return s._b(o, lambda o: GF(s.v * o._inv()), "/")
    def __rtruediv__(s, o): return s._b(o, lambda o: GF(o.v * s._inv()), "/", True)
    def __pow__(s, n):
        if isinstance(n, (GF, Opq)) or int(n) != n:
            if _BR[0] is None: raise ValueBranch("GF: 정수가 아닌 지수")
            return Opq()                                # 실수 거듭제곱(걸음 제어기 등): 비대수 값
        n = int(n); r = GF(pow(s.v, n, P)) if n >= 0 else GF(pow(s._inv(), -n, P))
        if _LIN[0] and s.l is not None:
            if n == 1: r.l = dict(s.l)
            elif _pure(s.l) or n == 0: r.l = {None: r.v} if r.v else {}
            else: r.l = {"~": frozenset(_vars(s.l))}
        return r
    def __neg__(s): return GF(-s.v, None if s.l is None else {x: (c if x == "~" else -c % P) for x, c in
 s.l.items()})
    def __pos__(s): return s
    def __eq__(s, o):
        """대상 코드의 값 같음 비교는 분기이다: 무작위 값에서는 거의 늘 거짓이라 '같음' 쪽 층을 덮지 못
한다(21.3절).
        결정 열거 중이면 경로 조건을 풀어 두 갈래를 모두 열고(Sched.eq), 아니면 범위 밖으로 멈춘다.
        검증기 자신은 .v를 직접 비교한다."""
        if _BR[0] is None: raise ValueBranch("값의 같음 비교(같음 층은 무작위 값이 덮지 못한다)")
        d = s - o
        return NotImplemented if d is NotImplemented else _BR[0].eq(d, False)
    def __ne__(s, o):
        if _BR[0] is None: raise ValueBranch("값의 같음 비교(같음 층은 무작위 값이 덮지 못한다)")
        d = s - o
        return NotImplemented if d is NotImplemented else _BR[0].eq(d, True)
    def __hash__(s): return hash(s.v)
    def _cmp(s, o):
        if _BR[0] is None: raise ValueBranch("값에 대한 비교(GF에는 순서가 없다)")
        return _BR[0].decide("GF 비교")
    __lt__ = __le__ = __gt__ = __ge__ = _cmp
    def __abs__(s):
        if _BR[0] is None: raise ValueBranch("abs: 값에 따라 갈리는 분기")
        return Opq()
    def __float__(s): raise ValueBranch("GF -> float")
    def __bool__(s): raise ValueBranch("GF의 참거짓")
    def __repr__(s): return f"GF({s.v})"


def gf_rand(rng): return GF(rng.randrange(1, P))


# ---------------------------------------------------------------------- (D) 값 분기: 결정 열거
_BR = [None]


class NeedMore(Exception): pass


class Pin(Exception):
    """같음 갈래를 연다: 경로 조건 d = 0을 d에 1차로 나온 변수 하나에 대해 풀었다.  var := form.
    d 전체가 1차식이면 form은 남은 자유 변수의 1차식이고 그 변수들은 계속 자유이다(V를 매개화).
    d에 1차가 아닌 부분이 있으면 d가 기대는 다른 변수들을 지금 값으로 고정하고(extra), var는 그 값들로
푼 상수이다
    — 다른 변수는 무작위로 뽑힌 값 그대로이므로 여전히 V 위의 무작위 점이다."""
    def __init__(s, var, form, extra=None): s.var, s.form, s.extra = var, form, dict(extra or {})


def add_pin(pins, e): return {**pins, **e.extra, e.var: e.form}


class Sched:
    """결정 열의 앞부분(prefix)을 따라 실행한다.  prefix를 넘는 결정이 오면 NeedMore로 멈춰 두 갈래를 연
다.
    pins: 지금 경로의 같음 조건으로 고정한 입력 변수 {번호: 자유 변수의 1차식} — 같음 갈래는 그 조건이
정의하는
    집합 V 위의 무작위 점(자유 변수만 무작위)에서 판정한다(21.3절)."""
    def __init__(s, prefix, rng, pins=None):
        s.prefix, s.pos, s.rng, s.fixed, s.log, s.pins = prefix, 0, rng, 0, [], dict(pins or {})
        s.fcount, s.val = 0, {}                         # f의 새 값 번호, 자유 변수의 현재 값
```

```
    def decide(s, why):
        if s.pos >= len(s.prefix): raise NeedMore()
        b = s.prefix[s.pos]; s.pos += 1; s.log.append(b); return b
    def eq(s, d, neg):
        """d = 왼쪽 - 오른쪽.  반환: (d == 0) xor neg.
        d가 상수식이면 결정이 아니다(참·거짓이 정해짐).  아니면 한 비트: 거짓 갈래는 무작위 점 그대로(어긋날 확률 <= 1/p),
        참 갈래는 d = 0을 변수 하나에 대해 풀어 Pin을 던진다 — 고정한 뒤 다시 돌리면 d는 상수 0이 된다."""
        if not PIN[0]:
            b = s.decide("GF 같음"); return b
        L = d.l
        if L is not None and _pure(L):
            s.fixed += 1; r = not L.get(None, 0); s.log.append(r); return r != neg
        if s.pos >= len(s.prefix): raise NeedMore()
        b = s.prefix[s.pos]; s.pos += 1
        if b:
            cands = [] if L is None else [x for x in L if x is not None and x != "~"]
            if not cands: raise ValueBranch("같음 갈래의 조건을 1차로 풀 변수가 없다(모든 변수가 비선형으로만 나온다) — 경로 조건을 풀 수 없다")
            var = min(cands, key=lambda x: (isinstance(x, tuple), str(x))); inv = pow(L[var], P - 2, P)
            if "~" not in L: raise Pin(var, {x: -c * inv % P for x, c in L.items() if x != var})
            raise Pin(var, {None: (s.val[var] - d.v * inv) % P}, {x: {None: s.val[x]} for x in _vars(L) - {var}})
        s.log.append(False); return neg


def bind_inputs(args, pins, sched=None):
    """args 안의 GF 값(배열·리스트·튜플·사전 속까지)에 입력 변수 번호를 매기고, 고정된 변수는 1차식의 값으로 바꾼다."""
    leaves = []
    def walk(a):
        if isinstance(a, GF): leaves.append(a)
        elif isinstance(a, np.ndarray) and a.dtype == object:
            for x in a.ravel(): walk(x)
        elif isinstance(a, (list, tuple)):
            for x in a: walk(x)
        elif isinstance(a, dict):
            for x in a.values(): walk(x)
    walk(args)
    val = {i: g.v for i, g in enumerate(leaves)}
    for i in val:
        if i in pins and _pure(pins[i]): val[i] = pins[i].get(None, 0)
    for i, g in enumerate(leaves):
        if i in pins:
            F = pins[i]; g.v = val[i] = sum(c * (1 if x is None else val[x]) for x, c in F.items()) % P; g.l = dict(F)
        else: g.l = {i: 1}
    if sched is not None: sched.val.update(val)
    return leaves


def gf_ratstr(v):
    """GF 원소를 작은 유리수 a/b로 되읽는다(유리수 재구성; |a|, b < sqrt(p/2)).  못 하면 그대로."""
    r0, r1, t0, t1 = P, v % P, 0, 1; B = int((P // 2) ** 0.5); S = 10 ** 6
    while r1 > B: q = r0 // r1; r0, r1, t0, t1 = r1, r0 - q * r1, t1, t0 - q * t1
    if t1 == 0 or abs(t1) > S or abs(r1) > S: return f"(무작위 값 {v})"
    a, b = (r1, t1) if t1 > 0 else (-r1, -t1)
    return str(a) if b == 1 else f"{a}/{b}"


def pins_str(pins, leaves=None):
    if not pins: return ""
    nm = lambda x: f"f호출#{x[1]}[{x[2]}]" if isinstance(x, tuple) else f"입력#{x}"
    return ", ".join(nm(i) + " := " + (" + ".join(f"{gf_ratstr(c)}·{nm(x)}" if x is not None else gf_ratstr(c) for x, c in F.items()) or "0")
                     for i, F in sorted(pins.items(), key=lambda t: str(t[0])))


class Opq:
    """비대수 값(abs, 노름, 실수 거듭제곱 등의 결과).  임의의 실수로 본다: 대수 식에 다시 들어가면 새 무작위 GF 값
    (그 값마다 고정), 상수와 비교하면 결정(두 갈래 모두 탐색), 두 불투명 값끼리의 비교는 한쪽으로 고정한다."""
    __slots__ = ("g",)
    def __init__(s): s.g = None
    def as_gf(s):
        if s.g is None: s.g = gf_rand(_BR[0].rng)
        return s.g
    def _op(s, o, f):
```

```
265          if isinstance(o, GF): return f(s.as_gf(), o)
266          return Opq()
267      def __add__(s, o): return s._op(o, lambda a, b: a + b)
268      def __radd__(s, o): return s._op(o, lambda a, b: b + a)
269      def __sub__(s, o): return s._op(o, lambda a, b: a - b)
270      def __rsub__(s, o): return s._op(o, lambda a, b: b - a)
271      def __mul__(s, o): return s._op(o, lambda a, b: a * b)
272      __rmul__ = __mul__
273      def __truediv__(s, o): return s._op(o, lambda a, b: a / b)
274      def __rtruediv__(s, o): return s._op(o, lambda a, b: b / a)
275      def __pow__(s, o): return Opq()
276      def __rpow__(s, o): return Opq()
277      def __neg__(s): return Opq()
278      def __abs__(s): return Opq()
279      def _c(s, o):
280          if isinstance(o, Opq): _BR[0].fixed += 1; return True
281          return _BR[0].decide("불투명 값과 상수")
282      __lt__ = __le__ = __gt__ = __ge__ = _c
283      def __float__(s): raise ValueBranch("불투명 값 -> float")
284
285
286  class Oracle:
287      """무작위 신탁 f: R^d -> R^d (GF 위).  같은 인자에는 같은 값."""
288      def __init__(s, rng, d): s.rng, s.d, s.tab, s.calls = rng, d, {}, 0
289      def __call__(s, y):
290          key = tuple(GF.c(v).v for v in np.asarray(y, dtype=object).ravel())
291          if len(key) != s.d: raise IndexError(f"f의 인자 길이 {len(key)} != {s.d}")
292          s.calls += 1
293          if key not in s.tab:
294              vals = [gf_rand(s.rng) for _ in range(s.d)]
295              if _LIN[0] and _BR[0] is not None:          # f의 새 값도 자유 변수: 같음 갈래가 f의 값에 걸
    리면 그 값을 고정한다
296                  sc = _BR[0]; n = sc.fcount; sc.fcount += 1
297                  for c, g in enumerate(vals):
298                      vid = ("f", n, c)
299                      if vid in sc.pins:
300                          F = sc.pins[vid]; g.v = sum(cf * (1 if x is None else sc.val[x]) for x, cf in F.
    items()) % P; g.l = dict(F)
301                      else: g.l = {vid: 1}
302                      sc.val[vid] = g.v
303              s.tab[key] = vals
304          return np.array(s.tab[key], dtype=object)
305
306
307  def gfvec(v): return np.array([GF.c(a) for a in np.asarray(v, dtype=object).ravel()], dtype=object)
308
309
310  def gf_eq(a, b):
311      a = np.asarray(a, dtype=object).ravel(); b = np.asarray(b, dtype=object).ravel()
312      if a.shape != b.shape: return False, -1
313      for i, (x, y) in enumerate(zip(a, b)):
314          if GF.c(x).v != GF.c(y).v: return False, i
315      return True, None
316
317
318  # ---------------------------------------------------------------------- 추적 정수
319  class Tracer:
320      def __init__(s): s.ev = []          # (kind, const, roots)
321      def rec(s, kind, c, roots): s.ev.append((kind, c, roots))
322
323
324  class TInt(int):
325      """추적 정수.  roots: 출처('n', 'len', 루프 번호 'k:...'), off: 출처에 더한 상수,
326      kind: 'aff'(출처 + 상수), 'per'(아핀 값의 나머지: 주기 칸), 'non'(정수 나눗셈·배수·곱 등 비아핀).
327      비아핀 값이 상수나 다른 추적 정수와 비교되면 대표 집합 정리가 서지 않으므로 'nonaffine'을 기록한다."
    ""
328      def __new__(cls, v, tr, roots, off=0, kind="aff"):
329          o = int.__new__(cls, v); o.tr, o.roots, o.off, o.kind = tr, frozenset(roots), off, kind; return
    o
330      def _mk(s, v, o, off, kind=None):
331          roots = s.roots | (o.roots if isinstance(o, TInt) else frozenset())
332          if kind is None:
333              ks = {s.kind, o.kind if isinstance(o, TInt) else "aff"}
334              kind = "non" if "non" in ks else ("per" if "per" in ks else "aff")
335              if isinstance(o, TInt) and "per" in ks and "aff" in ks: kind = "non"
336          return TInt(v, s.tr, roots, off, kind)
337      def __add__(s, o):
338          if not isinstance(o, int): return NotImplemented
339          return s._mk(int(s) + int(o), o, s.off + (o.off if isinstance(o, TInt) else int(o)))
340      __radd__ = __add__
```

```
    def __sub__(s, o):
        if not isinstance(o, int): return NotImplemented
        return s._mk(int(s) - int(o), o, s.off - (o.off if isinstance(o, TInt) else int(o)))
    def __rsub__(s, o):
        if not isinstance(o, int): return NotImplemented
        return s._mk(int(o) - int(s), o, int(o) - s.off, "non" if s.kind != "aff" else "non")
    def __mul__(s, o):
        if not isinstance(o, int): return NotImplemented
        if not isinstance(o, TInt) and abs(int(o)) == 1: return s._mk(int(s) * int(o), o, s.off * int(o)
, "non" if int(o) < 0 else None)
        return s._mk(int(s) * int(o), o, 0, "non")
    __rmul__ = __mul__
    def __neg__(s): return TInt(-int(s), s.tr, s.roots, -s.off, "non")
    def __floordiv__(s, o): return s._mk(int(s) // int(o), o, 0, "non")
    def __rfloordiv__(s, o): return s._mk(int(o) // int(s), o, 0, "non")
    def __mod__(s, o):
        if isinstance(o, TInt) or s.kind != "aff": return s._mk(int(s) % int(o), o, 0, "non")
        s.tr.rec("mod", abs(int(o)), s.roots); return s._mk(int(s) % int(o), o, 0, "per")
    def __and__(s, o): return s._mk(int(s) & int(o), o, 0, "non")
    __rand__ = __and__
    def __rshift__(s, o): return s._mk(int(s) >> int(o), o, 0, "non")
    def __lshift__(s, o): return s._mk(int(s) << int(o), o, 0, "non")
    def __truediv__(s, o): s.tr.rec("nonaffine", 0, s.roots); return int(s) / o
    def __rtruediv__(s, o): return o / int(s)
    def __pow__(s, o): return s._mk(int(s) ** int(o), o, 0, "non")
    def _c(s, o, op):
        if isinstance(o, int):
            rel = isinstance(o, TInt)
            kinds = {s.kind} | ({o.kind} if rel else set())
            if "non" in kinds or (rel and "per" in kinds): s.tr.rec("nonaffine", 0, s.roots)
            c = (s.off - o.off) if rel else (int(o) - s.off)
            s.tr.rec("cmp", abs(c), s.roots | (o.roots if rel else frozenset()))
        return op(int(s), o if not isinstance(o, TInt) else int(o))
    def __lt__(s, o): return s._c(o, lambda a, b: a < b)
    def __le__(s, o): return s._c(o, lambda a, b: a <= b)
    def __gt__(s, o): return s._c(o, lambda a, b: a > b)
    def __ge__(s, o): return s._c(o, lambda a, b: a >= b)
    def __eq__(s, o): return s._c(o, lambda a, b: a == b) if isinstance(o, int) else NotImplemented
    def __ne__(s, o): return s._c(o, lambda a, b: a != b) if isinstance(o, int) else NotImplemented
    __hash__ = int.__hash__
    def __bool__(s):
        if s.kind == "non": s.tr.rec("nonaffine", 0, s.roots)
        s.tr.rec("cmp", abs(s.off), s.roots); return int(s) != 0


def traced_builtins(tr):
    """대상 모듈 안의 range, len, enumerate를 추적판으로.  len이 한 실행에서 서로 다른 값을 두 번 넘게
돌려주면(재귀로
    줄어드는 길이 등) 그 길이들은 입력 크기의 아핀 함수로 추적되지 않으므로 'nonaffine'을 기록한다."""
    lens = set()
    def t_range(*a):
        a = list(a)
        if not any(isinstance(v, TInt) for v in a): yield from builtins.range(*[int(v) for v in a]); ret
urn
        roots = frozenset().union(*[v.roots for v in a if isinstance(v, TInt)])
        kind = "non" if any(isinstance(v, TInt) and v.kind != "aff" for v in a) else "aff"
        lo, hi, st = (0, a[0], 1) if len(a) == 1 else (a[0], a[1], a[2] if len(a) > 2 else 1)
        if isinstance(st, TInt) or int(st) != 1: kind = "non"
        for i in builtins.range(int(lo), int(hi), int(st)):
            yield TInt(i, tr, {"k:" + "/".join(sorted(roots))}, 0, kind)     # 루프 번호: 새 출처
    lens_by = {}
    def t_len(x):
        """차원 꼬리표(dimroot)가 붙은 입력은 그 꼬리표를 출처로: 서로 다른 독립 차원(깊이, 폭)은 따로
추적한다."""
        n = builtins.len(x); root = getattr(x, "dimroot", "len"); ls = lens_by.setdefault(root, set());
ls.add(n)
        if builtins.len(ls) > 1: tr.rec("nonaffine", 0, frozenset({root}))
        return TInt(n, tr, {root}, 0)
    def t_enum(x, start=0):
        for i, v in builtins.enumerate(x, start): yield TInt(i, tr, {"k:enum"}, 0), v
    return dict(range=t_range, len=t_len, enumerate=t_enum)


# ---------------------------------------------------------------- 대표 입력 집합(술어 칸마다 하나 이
상)
def representatives(ev, nmin, lookback=0, extra=()):
    """기록된 술어의 상수 C와 주기 L(나머지·나눗셈·배수의 최소공배수)로 대표 크기를 만든다.
        { nmin .. nmin+L+lookback+2 } ∪ { c+d : c in C, -2 <= d <= L+lookback+2 } ∪ { M+1 .. M+L+lookbac
k+2 },
    M = max(C ∪ {nmin}).  칸(상수와의 대소, 주기 L의 나머지)이 모두 대표를 가지며, 꼬리 칸은 주기적이다.
```

```
    """
    C = set(); mods = [1]
    for kind, c, _ in ev:
        if kind == "cmp": C.add(c)
        elif kind == "mod" and c > 0: mods.append(c)
    L = reduce(lambda a, b: a * b // math.gcd(a, b), mods, 1)
    M = max(C | {nmin}); span = L + lookback + 2
    R = set(range(nmin, nmin + span + 1)) | set(range(M + 1, M + span + 1))
    for c in C:
        for d in range(-2, span + 1): R.add(c + d)
    R |= set(extra)
    return sorted(r for r in R if r >= nmin), dict(C=sorted(C), L=L, M=M,
                                                   flags=sorted({k for k, _, _ in ev if k == "nonaffine"
})))


# ---------------------------------------------------------------- 대상 함수의 '적힌 식' 판(상수 나눗셈을 정확히)
class _Exact(ast.NodeTransformer):
    """int / int 를 정확한 유리수로(_q), 부동소수 상수를 rat()으로 읽는다.  나머지는 그대로."""
    def visit_BinOp(s, n):
        s.generic_visit(n)
        if isinstance(n.op, ast.Div): return ast.copy_location(ast.Call(ast.Name("_qdiv", ast.Load()), [n.left, n.right], []), n)
        return n
    def visit_Constant(s, n):
        if isinstance(n.value, float): return ast.copy_location(ast.Call(ast.Name("_qf", ast.Load()), [ast.Constant(n.value)], []), n)
        return n


def _qdiv(a, b):
    if isinstance(a, (int, np.integer)) and isinstance(b, (int, np.integer)) and not isinstance(a, bool):
        return Fr(int(a), int(b))
    if isinstance(a, Fr) and isinstance(b, (int, Fr)): return a / b
    if isinstance(a, (int, Fr)) and isinstance(b, Fr): return Fr(a) / b
    return a / b


def _qf(c): return rat(c)


def exact_version(fn_src, name, env):
    """원래 소스(또는 변이체 소스)의 함수를 정확 산술판으로 컴파일해 돌려준다."""
    tree = ast.parse(textwrap.dedent(fn_src)); tree = _Exact().visit(tree); ast.fix_missing_locations(tree)
    g = dict(env); g.update(_qdiv=_qdiv, _qf=_qf, np=np)
    exec(compile(tree, f"<exact {name}>", "exec"), g)
    return g[name], g


def float_version(fn_src, name, env):
    g = dict(env); g.update(np=np)
    exec(compile(textwrap.dedent(fn_src), f"<float {name}>", "exec"), g)
    return g[name]


# ---------------------------------------------------------------- 의도한 식(대상 코드와 독립)
def rk_step(A, b, f, y, h):
    """계수표 (A, b)의 명시적 룽게–쿠타 한 걸음(정확 산술; A, b는 유리수)."""
    K = []
    for i in range(len(b)):
        z = y
        for j in range(i):
            if A[i][j] != 0: z = z + h * A[i][j] * K[j]
        K.append(f(z))
    out = y
    for i in range(len(b)):
        if b[i] != 0: out = out + h * b[i] * K[i]
    return out


TAB = {
    "RK4": ([[0, 0, 0, 0], [Fr(1, 2), 0, 0, 0], [0, Fr(1, 2), 0, 0], [0, 0, 1, 0]], [Fr(1, 6), Fr(1, 3), Fr(1, 3), Fr(1, 6)]),
    "SSPRK3": ([[0, 0, 0], [1, 0, 0], [Fr(1, 4), Fr(1, 4), 0]], [Fr(1, 6), Fr(1, 6), Fr(2, 3)]),
}


```

```
def intended_onestep(method):
    A, b = TAB[method]
    def rel(f, Y, h, k): return rk_step(A, b, f, Y[k], h)
    return rel, 1, len(b)          # 관계, 필요한 과거 값 수, f 호출 수/걸음


def intended_ab3_rk4start():
    A, b = TAB["RK4"]; al = [Fr(5, 12), Fr(-16, 12), Fr(23, 12)]
    def rel(f, Y, h, k):
        if k < 2: return rk_step(A, b, f, Y[k], h)
        return Y[k] + h * (al[2] * f(Y[k]) + al[1] * f(Y[k - 1]) + al[0] * f(Y[k - 2]))
    return rel, 3, 4


def graph_apply(kind, x, u):
    """주장한 공간 이산의 식 (G):  (L u)_i = (1/V_i) sum_{j~i} T_ij (u_j - u_i)  — 1차원(대상과 독립).""
"
    N = len(x)
    if kind == "FEM P1":
        V = [0] * N; T = {}
        for e in range(N - 1):
            he = x[e + 1] - x[e]; T[(e, e + 1)] = 1 / he; V[e] = V[e] + he / 2; V[e + 1] = V[e + 1] + he
 / 2
    else:
        h = (x[N - 1] - x[0]) / (N - 1)
        V = ([h / 2] + [h] * (N - 2) + [h / 2]) if kind == "FDM" else [h] * N
        T = {(i, i + 1): 1 / h for i in range(N - 1)}
    out = [0] * N
    for (i, j), t in T.items():
        out[i] = out[i] + t * (u[j] - u[i]); out[j] = out[j] + t * (u[i] - u[j])
    return [out[i] / V[i] for i in range(N)]


def graph_apply_2d_fdm(u, nx, ny, h):
    """2차원 FDM(꼭짓점, 영 유량 유령점, 균일 h): 축별 1차원 FDM의 크로네커 합."""
    Vx = lambda i: h / 2 if i in (0, nx - 1) else h
    Vy = lambda j: h / 2 if j in (0, ny - 1) else h
    out = []
    for j in range(ny):
        for i in range(nx):
            p = j * nx + i; s = 0
            for di in (-1, 1):
                if 0 <= i + di < nx: s = s + (u[p + di] - u[p]) / (h * Vx(i))
            for dj in (-1, 1):
                if 0 <= j + dj < ny: s = s + (u[p + dj * nx] - u[p]) / (h * Vy(j))
            out.append(s)
    return out


# ------------------------------------------------------------------ (G)+(T): 시간 스킴, 모든 n
def _patched(g, tr):
    old = {k: g.get(k, None) for k in ("range", "len", "enumerate")}
    g.update(traced_builtins(tr)); return old


def _restore(g, old):
    for k, v in old.items():
        if v is None: g.pop(k, None)
        else: g[k] = v


def gf_time_run(fn, g, rel, n, d, seed, nout=lambda n: n + 1, tr=None):
    """한 크기 n에서 걸음마다 Y[k+1] == rel(f, Y, h, k)를 GF로 비교.  반환 (ok, info)."""
    rng = random.Random(seed); f = Oracle(rng, d)
    y0 = np.array([gf_rand(rng) for _ in range(d)], dtype=object); h = gf_rand(rng)
    tr = tr or Tracer(); old = _patched(g, tr)
    try:
        Y = fn(f, y0, h, TInt(n, tr, {"n"}))
    except ValueBranch as e:
        return None, dict(n=n, why=f"범위 밖: {e}")
    except Exception as e:
        return False, dict(n=n, step=None, why=f"실행 실패: {type(e).__name__}: {e}")
    finally:
        _restore(g, old)
    Y = list(Y)
    if len(Y) != nout(n): return False, dict(n=n, step=None, why=f"출력 길이 {len(Y)} != {nout(n)}")
    for k in range(len(Y) - 1):
        try:
            exp = rel(f, Y, h, k)
        except ValueBranch as e:
```

```
            return None, dict(n=n, why=f"범위 밖: {e}")
        ok, comp = gf_eq(Y[k + 1], exp)
        if not ok: return False, dict(n=n, step=k, comp=comp, why="관계식 위반")
    return True, dict(n=n, steps=len(Y) - 1, calls=f.calls)


def close_time(src, name, env, rel, d=3, nmin=1, lookback=0, nout=lambda n: n + 1, seeds=(1, 2), n_probe=12, confirm_min=False):
    """모든 n에서 '코드의 걸음 = 의도한 관계'를 판정.  반환 dict(verdict, ...)."""
    fn, g = exact_version(src, name, env)
    tr = Tracer()
    ok, info = gf_time_run(fn, g, rel, n_probe, d, seeds[0], nout, tr)     # 술어 기록용 첫 실행
    reps, meta = representatives(tr.ev, nmin, lookback)
    if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
    # 대표를 모두 돌리며 새 술어가 나오면 대표를 다시 만든다(고정점)
    seen = set(); fails = []
    while True:
        new = [r for r in reps if r not in seen]
        if not new: break
        for r in new:
            seen.add(r); tr2 = Tracer()
            for sd in seeds:
                ok, info = gf_time_run(fn, g, rel, r, d, sd * 1000003 + r, nout, tr2)
                if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
                if not ok: fails.append(info); break
            tr.ev += tr2.ev
        reps2, meta = representatives(tr.ev, nmin, lookback)
        reps = sorted(set(reps) | set(reps2))
    if fails:
        nstar = min(f_["n"] for f_ in fails)
        for r in (range(nmin, nstar) if confirm_min else ()):   # 선택: 그 아래를 모두 돌려 최소임을 따로 확인
            ok, info = gf_time_run(fn, g, rel, r, d, 7 * r + 5, nout)
            if ok is False: fails.append(info); nstar = r; break
        first = min((f_ for f_ in fails if f_["n"] == nstar), key=lambda f_: f_["step"] if f_["step"] is not None else -1)
        return dict(verdict="반례", info=first, meta=meta, reps=len(seen))
    if meta["flags"]:                                   # 비아핀 술어: 대표 집합 정리가 서지 않는다 -> 상한까지 전수
        bound = max(seen)
        for r in range(nmin, bound + 1):
            if r in seen: continue
            ok, info = gf_time_run(fn, g, rel, r, d, 13 * r + 1, nout)
            if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
            if ok is False: return dict(verdict="반례", info=info, meta=meta, reps=len(seen))
        return dict(verdict="유한 증명", meta=meta, bound=bound, reps=bound - nmin + 1)
    return dict(verdict="증명", meta=meta, reps=len(seen), max_n=max(seen),
                p_err=f"<= {len(seeds)}개 독립 시행이 모두 틀릴 확률 <= (deg/p)^{len(seeds)}, deg <= 4·걸음 수")


# ---------------------------------------------------------------- (G)+(T): 공간 연산자, 모든 N
def gf_space_run(fn, g, kind, N, seed, tr=None, uniform=False):
    rng = random.Random(seed)
    if uniform:
        x0, hh = gf_rand(rng), gf_rand(rng); x = [x0 + hh * i for i in range(N)]
    else:
        x = [gf_rand(rng) for _ in range(N)]
    u = [gf_rand(rng) for _ in range(N)]
    x = np.array(x, dtype=object); u_in = np.array(u, dtype=object)     # 실제 호출처럼 배열로(대상이 배열 연산을 써도 되게)
    tr = tr or Tracer(); old = _patched(g, tr)
    try:
        out = fn(list(u_in), x.copy())
    except ValueBranch as e:
        return None, dict(N=N, why=f"범위 밖: {e}")
    except Exception as e:
        return False, dict(N=N, node=None, why=f"실행 실패: {type(e).__name__}: {e}")
    finally:
        _restore(g, old)
    exp = graph_apply(kind, list(x), u)
    out = list(out)
    if len(out) != N: return False, dict(N=N, node=None, why=f"출력 길이 {len(out)} != {N}")
    for i in range(N):
        if GF.c(out[i]).v != GF.c(exp[i]).v: return False, dict(N=N, node=i, why="관계식 위반")
    return True, dict(N=N)


def gf_space2d_run(fn, g, nx, ny, seed, tr=None):
    rng = random.Random(seed); h = gf_rand(rng); u = [gf_rand(rng) for _ in range(nx * ny)]
```

```
    tr = tr or Tracer(); old = _patched(g, tr)
    try:
        out = fn(u, TInt(nx, tr, {"nx"}), TInt(ny, tr, {"ny"}), h)
    except ValueBranch as e:
        return None, dict(N=(nx, ny), why=f"범위 밖: {e}")
    except Exception as e:
        return False, dict(N=(nx, ny), node=None, why=f"실행 실패: {type(e).__name__}: {e}")
    finally:
        _restore(g, old)
    exp = graph_apply_2d_fdm(u, nx, ny, h); out = list(out)
    if len(out) != nx * ny: return False, dict(N=(nx, ny), node=None, why=f"출력 길이 {len(out)}")
    for i in range(nx * ny):
        if GF.c(out[i]).v != GF.c(exp[i]).v: return False, dict(N=(nx, ny), node=i, why="관계식 위반")
    return True, dict(N=(nx, ny))


def close_space(src, name, env, kind, nmin=2, seeds=(1, 2), n_probe=9, two_d=False):
    fn, g = exact_version(src, name, env); uniform = kind in ("FDM", "FVM")
    run = (lambda r, sd, tr: gf_space2d_run(fn, g, r[0], r[1], sd, tr)) if two_d else \
          (lambda r, sd, tr: gf_space_run(fn, g, kind, r, sd, tr, uniform))
    probe = (n_probe, n_probe - 1) if two_d else n_probe
    tr = Tracer(); ok, info = run(probe, seeds[0], tr)
    if ok is None: return dict(verdict="범위 밖", info=info)

    def repset(ev):
        if not two_d: return representatives(ev, nmin)
        r1, m1 = representatives(ev, nmin); return [(a, b) for a in r1 for b in r1], m1
    reps, meta = repset(tr.ev); seen = set(); fails = []
    while True:
        new = [r for r in reps if r not in seen]
        if not new: break
        for r in new:
            seen.add(r); tr2 = Tracer()
            for sd in seeds:
                ok, info = run(r, sd * 7919 + (r if not two_d else r[0] * 1000 + r[1]), tr2)
                if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
                if not ok: fails.append(info); break
            tr.ev += tr2.ev
        reps2, meta = repset(tr.ev); reps = sorted(set(reps) | set(reps2))
    if fails:
        key = (lambda f_: f_["N"]) if not two_d else (lambda f_: (sum(f_["N"]), f_["N"]))
        first = min(fails, key=key)
        if not two_d:
            for r in range(nmin, first["N"]):
                ok, info = run(r, 31 * r + 3, None)
                if ok is False: first = info; break
        return dict(verdict="반례", info=first, meta=meta, reps=len(seen))
    if meta["flags"]:
        bound = max(seen) if not two_d else max(max(r) for r in seen)
        allr = [(p_, q_) for p_ in range(nmin, bound + 1) for q_ in range(nmin, bound + 1)] if two_d els
e range(nmin, bound + 1)
        for r in allr:
            if r in seen: continue
            ok, info = run(r, 17, None)
            if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
            if ok is False: return dict(verdict="반례", info=info, meta=meta, reps=len(seen))
        return dict(verdict="유한 증명", meta=meta, bound=bound)
    return dict(verdict="증명", meta=meta, reps=len(seen))


# ------------------------------------------------------------------ (S): 결정적 기호 판정
def canon(e):
    """유리 연산 + 해석하지 않은 함수의 정규형: 함수 인자를 안쪽부터 정규화한 뒤 전체를 cancel."""
    e = sp.sympify(e)
    if e.is_Atom: return e
    if isinstance(e, sp.core.function.AppliedUndef):
        return e.func(*[canon(a) for a in e.args])
    return sp.cancel(e.func(*[canon(a) for a in e.args]))


def sym_vec(name, d): return np.array(sp.symbols(f"{name}0:{d}"), dtype=object)


def sym_f(d):
    Fs = [sp.Function(f"f{i}") for i in range(d)]
    return lambda y: np.array([Fi(*[canon(v) for v in np.asarray(y, dtype=object).ravel()]) for Fi in Fs
], dtype=object)


def sym_time_check(src, name, env, rel, n, d=2):
```

```
    """크기 n 하나에서, 모든 f(해석하지 않은 함수)·모든 y0·모든 h에 대해 걸음마다 Y[k+1] == rel 을 결정적으로 판정."""
    fn, g = exact_version(src, name, env); f = sym_f(d); y0 = sym_vec("y", d); h = sp.Symbol("h")
    Y = list(fn(f, y0, h, n))
    for k in range(len(Y) - 1):
        exp = rel(f, Y, h, k)
        for c in range(d):
            if sp.simplify(canon(np.asarray(Y[k + 1]).ravel()[c] - np.asarray(exp).ravel()[c])) != 0:
                return False, dict(step=k, comp=c)
    return True, dict(steps=len(Y) - 1)


def sym_space_check(src, name, env, kind, N, uniform=False):
    fn, g = exact_version(src, name, env)
    if uniform:
        x0, hh = sp.symbols("x0 h"); x = [x0 + hh * i for i in range(N)]
    else:
        x = list(sp.symbols(f"x0:{N}"))
    u = list(sp.symbols(f"u0:{N}"))
    out = fn(list(u), np.array(x, dtype=object)); exp = graph_apply(kind, x, u)
    for i in range(N):
        if sp.cancel(sp.together(out[i] - exp[i])) != 0: return False, dict(node=i)
    return True, dict(N=N)


# ---------------------------------------------------------------- (B): 나무 ODE로 코드 자체의 차수
def tree_ode(t):
    """뿌리 나무 t(claim_tools의 표현: 자식 목록의 리스트, 잎은 [])의 꼭짓점별 다항 ODE.  반환 (F, d, |t|, gamma)."""
    verts = []
    def walk(node):
        i = len(verts); verts.append(None); ch = [walk(c) for c in node]; verts[i] = ch; return i
    walk(t); d = len(verts)
    def F(y):
        y = list(np.asarray(y, dtype=object).ravel())
        out = []
        for ch in verts:
            p = 1
            for c in ch: p = p * y[c]
            out.append(p)
        return np.array(out, dtype=object)
    def gam(node):
        n = 1 + sum(size(c) for c in node); return n * math.prod(gam(c) for c in node)
    def size(node): return 1 + sum(size(c) for c in node)
    return F, d, size(t), gam(t)


def rooted_trees(p):
    """차수 1..p의 뿌리 나무(중복 없는 표현)."""
    T = {1: [()]}
    for n in range(2, p + 1):                # 크기 n-1의 숲(나무의 다중집합)을 뿌리 아래에
        T[n] = sorted(set(tuple(sorted(f_)) for f_ in _forests(n - 1, T)))
    return T


def _forests(n, T):
    """크기 합이 n인 나무들의 다중집합(정렬된 튜플)."""
    if n == 0: yield (); return
    allt = [(m, t) for m in range(1, n + 1) for t in T.get(m, [])]
    def rec(rem, start):
        if rem == 0: yield (); return
        for idx in range(start, len(allt)):
            m, t = allt[idx]
            if m <= rem:
                for rest in rec(rem - m, idx): yield (t,) + rest
    yield from rec(n, 0)


def code_order(src, name, env, pmax=5, run_steps=1):
    """코드(한 걸음 방법의 driver; n=1로 돌린다)의 기본 무게를 나무 ODE로 정확히 읽어 차수를 판정.
    반환 (차수, 위반 목록[(나무, 크기, 코드 계수, 1/gamma)])."""
    fn, g = exact_version(src, name, env); h = sp.Symbol("h"); T = rooted_trees(pmax); bad = []
    for n in range(1, pmax + 1):
        for t in T[n]:
            F, d, sz, gm = tree_ode(t)
            y0 = np.array([sp.Integer(0)] * d, dtype=object)
            Y = list(fn(F, y0, h, 1)); yr = sp.expand(np.asarray(Y[1], dtype=object).ravel()[0])
            c = sp.Poly(yr, h).coeff_monomial(h ** sz) if yr != 0 else 0
            if sp.nsimplify(c) != sp.Rational(1, gm): bad.append((t, sz, sp.nsimplify(c), sp.Rational(1, gm)))
```

```
    order = min([b[1] for b in bad], default=pmax + 1) - 1
    return order, bad


def tree_str(t):
    return "τ" if t == () else "[" + ",".join(tree_str(c) for c in t) + "]"


def close_branching(src, name, env, check, d=3, D=16, seed=1, inputs=None):
    """값 분기가 있는 코드: 결정 D번 이내로 끝나는 모든 경로를 깊이 우선으로 열거하고, 경로마다 GF로 check(출력, f)를 판정.
    같음 갈래는 경로 조건(입력과 f의 값에 대한 1차식)을 풀어 대입하고, 그 조건 위의 점에서 판정한다(실현 불가 경로를 판정하지 않는다).
    풀 수 없는 같음 갈래(비선형)는 그 경로만 '범위 밖'으로 남기고 나머지 경로를 계속 연다.
    반환 dict(verdict, paths, truncated, info)."""
    fn, g = exact_version(src, name, env); stack = [([], {})]; paths = trunc = 0; fixed = 0; unres = None
    _LIN[0] = True
    try:
        while stack:
            pre, pins = stack.pop()
            rng = random.Random(seed * 7777 + len(pre) * 31 + sum(b << i for i, b in enumerate(pre)))
            f = Oracle(rng, d); args = inputs(rng) if inputs else ()
            _BR[0] = sc_ = Sched(pre, rng, pins); bind_inputs(args, pins, sc_)
            try:
                out = fn(f, *args)
            except NeedMore:
                if len(pre) >= D: trunc += 1
                else: stack += [(pre + [False], pins), (pre + [True], pins)]
                continue
            except Pin as e:
                stack.append((pre[:-1], add_pin(pins, e))); continue
            except ValueBranch as e:
                if "같음 갈래" in str(e): unres = unres or str(e); continue
                return dict(verdict="범위 밖", info=str(e), paths=paths)
            except Exception as e:
                return dict(verdict="반례", info=dict(path=sc_.log, pins=pins_str(pins), why=f"실행 실패: {type(e).__name__}: {e}"), paths=paths)
            finally:
                fixed += sc_.fixed; _BR[0] = None
            _BR[0] = sc_
            try:
                ok, info = check(out, f, args)
            except Exception as e:
                ok, info = False, dict(why=f"출력 검사 실패: {type(e).__name__}: {e}")
            finally:
                _BR[0] = None
            paths += 1
            if not ok: return dict(verdict="반례", info=dict(path=sc_.log, pins=pins_str(pins), **info), paths=paths)
    finally:
        _BR[0] = None; _LIN[0] = False
    if unres: return dict(verdict="범위 밖", info=unres, paths=paths)
    return dict(verdict="증명", paths=paths, truncated=trunc, D=D, fixed=fixed)


from contextlib import contextmanager


@contextmanager
def field(p):
    """GF의 법을 잠시 바꾼다(NTT 소수, secp256k1의 체 등)."""
    global P
    old = P; P = p
    try: yield
    finally: P = old
```

작동 확인 targets.py — 20.6–20.7절의 대상 코드와 숨긴 조건부 버그

```
"""targets: 오라클 실험(20절)의 대상 코드 — 사용자가 짤 법한 평범한 구현.  검증기는 이 코드의 내부를 보지 않고,
부동소수·기호·GF(p) 어느 산술로든 그대로 돌린다.  변이체는 이 소스에서 한 자리씩 바꿔 만든다(mutate.py).
시간: f(y) -> 우변, y0, h, n -> 상태 n+1개.   공간: (u, x) -> L u   (2차원은 (u, nx, ny, h)).
각 대상의 주장(의도한 식)은 oracle_close의 계수표·그래프 식이 독립으로 맡는다."""

RK4 = '''
def rk4_run(f, y0, h, n):
    Y = [y0]
    y = y0
```

```
    for k in range(n):
        k1 = f(y)
        k2 = f(y + h / 2 * k1)
        k3 = f(y + h / 2 * k2)
        k4 = f(y + h * k3)
        y = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        Y.append(y)
    return Y
'''

SSPRK3 = '''
def ssprk3_run(f, y0, h, n):
    Y = [y0]
    y = y0
    for k in range(n):
        u1 = y + h * f(y)
        u2 = 3 / 4 * y + 1 / 4 * (u1 + h * f(u1))
        y = 1 / 3 * y + 2 / 3 * (u2 + h * f(u2))
        Y.append(y)
    return Y
'''

AB3 = '''
def ab3_run(f, y0, h, n):
    Y = [y0]
    for k in range(2):
        y = Y[-1]
        k1 = f(y)
        k2 = f(y + h / 2 * k1)
        k3 = f(y + h / 2 * k2)
        k4 = f(y + h * k3)
        Y.append(y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4))
    F = [f(v) for v in Y]
    for k in range(2, n):
        Y.append(Y[-1] + h * (23 / 12 * F[-1] - 16 / 12 * F[-2] + 5 / 12 * F[-3]))
        F.append(f(Y[-1]))
    return Y
'''

FEM = '''
def fem_p1_apply(u, x):
    N = len(x)
    Lu = [0] * N
    for e in range(N - 1):
        he = x[e + 1] - x[e]
        flux = (u[e + 1] - u[e]) / he
        Lu[e] = Lu[e] + flux
        Lu[e + 1] = Lu[e + 1] - flux
    for i in range(N):
        if i == 0:
            V = (x[1] - x[0]) / 2
        elif i == N - 1:
            V = (x[N - 1] - x[N - 2]) / 2
        else:
            V = (x[i + 1] - x[i - 1]) / 2
        Lu[i] = Lu[i] / V
    return Lu
'''

FVM = '''
def fvm_apply(u, x):
    N = len(x)
    h = (x[N - 1] - x[0]) / (N - 1)
    Lu = [0] * N
    for i in range(N - 1):
        F = (u[i + 1] - u[i]) / h
        Lu[i] = Lu[i] + F / h
        Lu[i + 1] = Lu[i + 1] - F / h
    return Lu
'''

FDM2D = '''
def fdm2d_apply(u, nx, ny, h):
    Lu = [0] * (nx * ny)
    for j in range(ny):
        for i in range(nx):
            p = j * nx + i
            c = u[p]
            if i == 0:
                ax = 2 * (u[p + 1] - c)
```

```
            elif i == nx - 1:
                ax = 2 * (u[p - 1] - c)
            else:
                ax = u[p + 1] - 2 * c + u[p - 1]
            if j == 0:
                ay = 2 * (u[p + nx] - c)
            elif j == ny - 1:
                ay = 2 * (u[p - nx] - c)
            else:
                ay = u[p + nx] - 2 * c + u[p - nx]
            Lu[p] = (ax + ay) / h ** 2
    return Lu
'''

# ---------------------------------------------------------------- 조건부 버그(손으로 넣은 것): 표준 시험이 놓치게 만든 자리
COND = {
    # 문서 19.3절의 것과 같은 종류: 걸음 수가 7의 배수일 때만 마지막 걸음에서 k3 자리에 k2를 쓴다
    "RK4: 걸음 수가 7의 배수일 때 마지막 걸음": ("rk4", RK4.replace(
        "        k4 = f(y + h * k3)\n",
        "        if n % 7 == 0 and k == n - 1:\n            k3 = k2\n        k4 = f(y + h * k3)\n")),
    # 긴 계산에서만: 1000번째 걸음부터 AB3의 계수 하나가 틀린다(재시작 코드를 잘못 붙인 경우)
    "AB3: 1000걸음 뒤부터 계수 오타": ("ab3", AB3.replace(
        "        Y.append(Y[-1] + h * (23 / 12 * F[-1] - 16 / 12 * F[-2] + 5 / 12 * F[-3]))\n",
        "        c0 = 23 / 12 if k < 1000 else 22 / 12\n"
        "        Y.append(Y[-1] + h * (c0 * F[-1] - 16 / 12 * F[-2] + 5 / 12 * F[-3]))\n")),
    # 문서 19.3절: 크기에 따라 숨은 버그 — 마디가 40개를 넘으면 가운데 마디의 유량 하나를 두 번 더한다
    "FEM P1: 마디 40개 초과일 때 가운데 마디": ("fem", FEM.replace(
        "        Lu[e + 1] = Lu[e + 1] - flux\n",
        "        Lu[e + 1] = Lu[e + 1] - flux\n        if N > 40 and e == N // 2:\n            Lu[e] = Lu[e] + flux\n")),
    # 문서 19.3절: 비균일 격자에서만 틀림 — 칸 부피를 평균 간격으로
    "FEM P1: 비균일 격자에서만(평균 간격 부피)": ("fem", FEM.replace(
        "            V = (x[i + 1] - x[i - 1]) / 2\n",
        "            V = (x[N - 1] - x[0]) / (N - 1)\n")),
    # 2차원: 정사각 격자(nx == ny)일 때만 맞고 직사각이면 y 경계를 nx로 잡는다
    "FDM 2D: 직사각 격자에서만 위쪽 경계": ("fdm2d", FDM2D.replace(
        "            elif j == ny - 1:\n", "            elif j == nx - 1:\n")),
}

ALL = dict(rk4=("rk4_run", RK4), ssprk3=("ssprk3_run", SSPRK3), ab3=("ab3_run", AB3),
           fem=("fem_p1_apply", FEM), fvm=("fvm_apply", FVM), fdm2d=("fdm2d_apply", FDM2D))

# ---------------------------------------------------------------- 값 분기가 있는 대상: 적응형 보가키-샴핀 3(2), FSAL
BS23 = '''
def bs23_run(f, y0, h, T, tol):
    t = 0
    y = y0
    ts = [t]
    Y = [y0]
    k1 = f(y)
    while t < T:
        if t + h > T:
            h = T - t
        k2 = f(y + h / 2 * k1)
        k3 = f(y + 3 / 4 * h * k2)
        ynew = y + h * (2 / 9 * k1 + 1 / 3 * k2 + 4 / 9 * k3)
        k4 = f(ynew)
        e = h * (-5 / 72 * k1 + 1 / 12 * k2 + 1 / 9 * k3 - 1 / 8 * k4)
        err = max(abs(v) for v in e)
        if err <= tol:
            t = t + h
            y = ynew
            k1 = k4
            ts.append(t)
            Y.append(y)
        h = 9 / 10 * h * (tol / max(err, tol / 1000)) ** (1 / 3)
    return ts, Y
'''

BRANCH_BUGS = {
    # 거절된 걸음 뒤에도 FSAL 값을 넘긴다: 거절이 한 번이라도 일어나는 실행에서만 틀린다
    "BS23: 거절 뒤 k1 = k4(FSAL 오용)": BS23.replace(
        "        h = 9 / 10 * h", "        else:\n            k1 = k4\n        h = 9 / 10 * h"),
    # 마지막 걸음을 T에 맞추는 자리의 오타: 끝을 넘는 시도가 있을 때만
    "BS23: 끝 걸음 맞춤 오타(T - h)": BS23.replace("            h = T - t\n", "            h = T - h\n"),
    # 가중치 오타(문서 A5의 bs23bug와 같은 종류): 모든 걸음
    "BS23: 가중치 1/3 -> 1/3 + 1/50": BS23.replace("1 / 3 * k2 + 4 / 9", "(1 / 3 + 1 / 50) * k2 + 4 / 9"
```

```
    ),
166 }
```

작동 확인 mutate.py — 변이 연산자 다섯 가지

```
  1 """mutate: 대상 소스에서 한 자리만 바꾼 변이체를 모두 만든다(20.4절).  변이 연산자는 소프트웨어 시험의
    표준
  2 변이 분석에서 쓰는 다섯 가지이다.
  3   AOR  산술 연산자 바꾸기   + - * / 사이
  4   CRP  정수 상수 바꾸기     c -> c+1, c-1
  5   ROR  비교 연산자 바꾸기   == != < <= > >= 사이
  6   SVR  변수 바꾸기          읽는 자리의 이름을 같은 함수 안의 다른 지역 이름으로(복사·붙여넣기 오류)
  7   IDX  첨자 밀기            a[i] -> a[i+1], a[i-1]
  8 같은 소스가 되는 변이체는 하나로 친다."""
  9 import ast, copy
 10
 11
 12 class _Sites(ast.NodeVisitor):
 13     def __init__(s): s.sites = []; s.names = set(); s.params = set()
 14     def visit_FunctionDef(s, n):
 15         s.params |= {a.arg for a in n.args.args}; s.generic_visit(n)
 16     def visit_Name(s, n):
 17         if isinstance(n.ctx, ast.Store): s.names.add(n.id)
 18         s.generic_visit(n)
 19
 20
 21 def _nodes(tree):
 22     return list(ast.walk(tree))
 23
 24
 25 OPS = [ast.Add, ast.Sub, ast.Mult, ast.Div]
 26 CMP = [ast.Eq, ast.NotEq, ast.Lt, ast.LtE, ast.Gt, ast.GtE]
 27
 28
 29 def mutants(src):
 30     base = ast.parse(src); orig = ast.unparse(base)
 31     sv = _Sites(); sv.visit(base)
 32     pool = sorted((sv.names | sv.params) - {"f"})
 33     out = {}
 34     nodes = _nodes(base)
 35     for idx, node in enumerate(nodes):
 36         edits = []
 37         if isinstance(node, ast.BinOp) and type(node.op) in OPS:
 38             for op in OPS:
 39                 if op is not type(node.op): edits.append(("AOR", lambda n, op=op: setattr(n, "op", op())
    ))
 40         if isinstance(node, ast.Constant) and isinstance(node.value, int) and not isinstance(node.value,
     bool):
 41             for d in (1, -1):
 42                 edits.append(("CRP", lambda n, d=d: setattr(n, "value", n.value + d)))
 43         if isinstance(node, ast.Compare) and len(node.ops) == 1:
 44             for op in CMP:
 45                 if op is not type(node.ops[0]): edits.append(("ROR", lambda n, op=op: setattr(n, "ops",
    [op()])))
 46         if isinstance(node, ast.Name) and isinstance(node.ctx, ast.Load) and node.id in pool:
 47             for nm in pool:
 48                 if nm != node.id: edits.append(("SVR", lambda n, nm=nm: setattr(n, "id", nm)))
 49         if isinstance(node, ast.Subscript) and not isinstance(node.slice, ast.Slice):
 50             for d in (1, -1):
 51                 edits.append(("IDX", lambda n, d=d: setattr(n, "slice", ast.BinOp(n.slice, ast.Add(), as
    t.Constant(d)))))
 52         for kind, fn in edits:
 53             t = copy.deepcopy(base); tgt = _nodes(t)[idx]; fn(tgt); ast.fix_missing_locations(t)
 54             try:
 55                 s = ast.unparse(t); compile(s, "<m>", "exec")
 56             except Exception:
 57                 continue
 58             if s != orig and s not in out: out[s] = (kind, ast.unparse(tgt)[:60], getattr(nodes[idx], "l
    ineno", None))
 59     return [(k, where, line, s) for s, (k, where, line) in out.items()]
```

작동 확인 oracle_experiment.py — 20.6절: 세 오라클의 변이 분석

```
"""oracle_experiment: 세 오라클을 같은 변이체 전부에 돌려 비교한다(20.4절).
  B0  제조해·수렴 차수(코드 검증의 표준 관행): 정확해가 있는 문제를 격자·걸음 4단계로 돌려 관측 차수 >= 공칭 - 0.3
  B1  이 문서 1–19절의 실행 검사: 사용자가 실제로 돌린 한 번의 실행에서 주장의 관계식(check_rk, check_lmm, space_check)
  B2  20절의 닫힘 검사: 모든 n, N, f, 상태, 격자에서 코드의 식 = 의도한 식 (oracle_close)
변이체가 B2를 통과하면, 그 변이체가 원래 코드와 같은 식인지를 결정적 기호 판정(S)으로 따로 확인한다(동치 확인).
결과는 oracle_results.json과 표준 출력의 표."""
import json, math, os, signal, sys, time, traceback, warnings
import multiprocessing as mp
import numpy as np
warnings.filterwarnings("ignore")
import oracle_close as oc, targets as T, mutate

NOM = dict(rk4=4, ssprk3=3, ab3=3, fem=2, fvm=2, fdm2d=2)
KIND = dict(fem="FEM P1", fvm="FVM")
TAB = {k: ([[float(a) for a in r] for r in v[0]], [float(b) for b in v[1]]) for k, v in oc.TAB.items()}


class Timeout(Exception): pass


def _alarm(sig, frm): raise Timeout()


def limited(fn, sec=60):
    signal.signal(signal.SIGALRM, _alarm); signal.alarm(sec)
    try: return fn()
    except Timeout: return ("timeout",)
    finally: signal.alarm(0)


def finite_list(Y):
    try: return all(np.all(np.isfinite(np.asarray(v, float))) for v in Y)
    except Exception: return False


# ------------------------------------------------------------------ B0
def ode_f(y): return np.array([y[0] * (1 - y[0]), -y[2], y[1]])
Y0 = np.array([0.1, 1.0, 0.0]); EX = np.array([1 / (1 + 9 * math.exp(-1)), math.cos(1), math.sin(1)])


def b0_time(fn, key):
    errs = []
    for n in (10, 20, 40, 80):
        Y = fn(ode_f, Y0.copy(), 1.0 / n, n)
        if len(Y) != n + 1 or not finite_list(Y): return True, f"n={n}: 출력 길이·유한성"
        errs.append(float(np.max(np.abs(np.asarray(Y[-1], float) - EX))))
    if errs[-1] < 1e-12: return False, "반올림 수준"
    p = math.log2(errs[-2] / errs[-1]) if errs[-1] > 0 else 99
    return (p < NOM[key] - 0.3, f"관측 차수 {p:.2f}")


def graded(n): s = np.linspace(0, 1, n); return s - 0.08 * np.sin(2 * np.pi * s) / (2 * np.pi) * 2


def b0_space(fn, key):
    if key == "fdm2d":
        errs = []
        for n in (6, 11, 21, 41):
            h = 1 / (n - 1); xs = np.linspace(0, 1, n); X, Yg = np.meshgrid(xs, xs)
            u = (np.cos(np.pi * X) * np.cos(np.pi * Yg)).ravel(); out = np.asarray(fn(list(u), n, n, h), float)
            if out.shape != u.shape or not np.all(np.isfinite(out)): return True, "출력"
            errs.append(np.max(np.abs(out + 2 * np.pi ** 2 * u)))
        p = math.log2(errs[-2] / errs[-1]) if errs[-1] > 0 else 99
        return (p < NOM[key] - 0.3, f"관측 차수 {p:.2f}")
    worst = 99
    for grid in ("uniform", "graded"):
        errs = []
        for n in (11, 21, 41, 81):
            x = np.linspace(0, 1, n) if grid == "uniform" else graded(n)
            if key == "fvm": x = (np.arange(n) + 0.5) / n
            u = np.cos(np.pi * x); out = np.asarray(fn(list(u), x.copy()), float)
            if out.shape != u.shape or not np.all(np.isfinite(out)): return True, "출력"
            errs.append(np.max(np.abs(out + np.pi ** 2 * u)))
        p = math.log2(errs[-2] / errs[-1]) if errs[-1] > 0 else 99; worst = min(worst, p)
```

```
        if key == "fvm": break
    return (worst < NOM[key] - 0.3, f"관측 차수 {worst:.2f}")


# ---------------------------------------------------------------- B1 (문서의 실행 검사)
_S = None


def sysS():
    global _S
    if _S is None:
        from schemes_lib import S; _S = S
    return _S


def b1_time(fn, key):
    from verify import check_rk, check_lmm
    S = sysS(); n = 200; Tn = 2.0; h = Tn / n
    Y = fn(S.F, S.u0.copy(), h, n)
    if len(Y) != n + 1 or not finite_list(Y): return True, "출력 길이·유한성"
    t = np.linspace(0, Tn, n + 1); Y = np.array([np.asarray(v, float) for v in Y])
    if key in ("rk4", "ssprk3"):
        r = check_rk(S, t, Y, *TAB["RK4" if key == "rk4" else "SSPRK3"])
        return (isinstance(r, tuple), f"걸음 {r[0]}" if isinstance(r, tuple) else "통과")
    r0 = check_rk(S, t[:3], Y[:3], *TAB["RK4"])
    if isinstance(r0, tuple): return True, f"출발 걸음 {r0[0]}"
    r = check_lmm(S, t, Y, [0, 0, -1, 1], [5 / 12, -16 / 12, 23 / 12, 0])
    return (isinstance(r, tuple), f"걸음 {r[0]}" if isinstance(r, tuple) else "통과")


FEMX = np.sort(np.concatenate([[0.0, 1.0], np.random.default_rng(5).uniform(0.02, 0.98, 19)]))


def b1_space(fn, key):
    import space_check as sc
    if key == "fdm2d":
        nx, ny = 7, 5; h = 1 / 6; xs, ys = np.arange(nx) * h, np.arange(ny) * h
        L = lambda e: fn(list(e), nx, ny, h); x = (xs, ys); kind = "FDM"; n = nx * ny
    else:
        x = FEMX.copy() if key == "fem" else (np.arange(20) + 0.5) / 20; kind = KIND[key]; n = len(x)
        L = lambda e: fn(list(e), x.copy())          # 변이체가 x를 제자리에서 바꿔도 다음 열·다음 변이
체에 새지 않게
    M = sc.as_matrix(L, n)
    if M.shape != (n, n) or not np.all(np.isfinite(M)): return True, "출력"
    r = sc.check(M, x, kind)
    return (not r["ok"], f"{r['viol'][0]['rule']} 마디 {r['viol'][0]['i']}" if not r["ok"] else "통과")


# ---------------------------------------------------------------- B2 (닫힘)
REL = dict(rk4=oc.intended_onestep("RK4")[0], ssprk3=oc.intended_onestep("SSPRK3")[0], ab3=oc.intended_a
b3_rk4start()[0])


def b2(src, key):
    name = T.ALL[key][0]
    if key in REL:
        r = oc.close_time(src, name, {}, REL[key], nmin=3 if key == "ab3" else 1, lookback=3 if key == "
ab3" else 0)
    elif key == "fdm2d":
        r = oc.close_space(src, name, {}, None, nmin=2, two_d=True, n_probe=5)
    else:
        r = oc.close_space(src, name, {}, KIND[key], nmin=2)
    v = r["verdict"]
    return v, (str(r.get("info", ""))[:120] if v != "증명" else f"대표 {r.get('reps')}개, 상수 {r['meta'
]['C']}, 주기 {r['meta']['L']}")


def equiv(src, key):
    """B2를 통과한 변이체: 결정적 기호 판정으로 원래 식과 같은지 확인."""
    name = T.ALL[key][0]
    if key in REL:
        return oc.sym_time_check(src, name, {}, REL[key], 3 if key == "ab3" else 2, d=1 if key == "ab3"
else 2)[0]
    if key == "fdm2d":
        import sympy as sp
        fn, g = oc.exact_version(src, name, {}); nx, ny = 4, 3; u = list(sp.symbols(f"u0:{nx * ny}")); h
 = sp.Symbol("h")
        out = fn(u, nx, ny, h); exp = oc.graph_apply_2d_fdm(u, nx, ny, h)
        return all(sp.cancel(sp.together(a - b)) == 0 for a, b in zip(out, exp))
    return oc.sym_space_check(src, name, {}, KIND[key], 6, uniform=key == "fvm")[0]
```

```

# ---------------------------------------------------------------- 한 변이체
def run_one(args):
    key, kind, where, line, src = args
    name = T.ALL[key][0]; res = dict(key=key, kind=kind, where=where, line=line, src=src)
    try:
        fn = oc.float_version(src, name, {})
    except Exception as e:
        res.update(B0=(True, "컴파일"), B1=(True, "컴파일"), B2=("반례", "컴파일")); return res
    for lab, f_ in (("B0", b0_time if key in REL else b0_space), ("B1", b1_time if key in REL else b1_sp
ace)):
        def go(f_=f_):
            with np.errstate(all="ignore"): return f_(fn, key)
        try:
            out = limited(go, 60)
            res[lab] = (True, "시간 초과") if out == ("timeout",) else out
        except Exception as e:
            res[lab] = (True, f"실행 실패 {type(e).__name__}")
    try:
        out = limited(lambda: b2(src, key), 300)
        res["B2"] = ("반례", "시간 초과") if out == ("timeout",) else out
    except Exception as e:
        res["B2"] = ("반례", f"실행 실패 {type(e).__name__}: {e}"[:120])
    res["crash"] = any("실행 실패" in str(res[k][1]) or "컴파일" in str(res[k][1]) for k in ("B0", "B1")
)
    if res["B2"][0] == "증명":
        try:
            res["equiv"] = bool(limited(lambda: equiv(src, key), 300))
        except Exception as e:
            res["equiv"] = f"오류 {e}"
    return res


def main():
    jobs = []
    for key, (name, src) in T.ALL.items():
        jobs.append((key, "원본", "", None, src))
        for kind, where, line, s in mutate.mutants(src): jobs.append((key, kind, where, line, s))
    for lab, (key, s) in T.COND.items(): jobs.append((key, "조건부:" + lab, "", None, s))
    if len(sys.argv) > 1: jobs = [j for j in jobs if j[0] in sys.argv[1:]]
    t0 = time.time(); out = []
    with mp.Pool(max(1, os.cpu_count())) as pool:
        for i, r in enumerate(pool.imap_unordered(run_one, jobs, chunksize=1)):
            out.append(r)
            if i % 100 == 0: print(f"{i}/{len(jobs)} {time.time() - t0:.0f}s", flush=True)
    json.dump(out, open("oracle_results.json", "w"), ensure_ascii=False, indent=0, default=str)
    print(f"끝 {time.time() - t0:.0f}s")


if __name__ == "__main__":
    main()
```

작동 확인 brute_check.py — 20.6절: 대표 집합의 전수 교차 확인

```
"""brute_check: 대표 입력 집합(T)의 교차 확인 — 대표만 본 B2 판정과, 작은 크기를 모두 돈 판정이 같은가(2
0.4절).
  B2가 '증명'이면  n(N) = nmin..상한 을 모두 돌려도 반례가 없어야 하고,
  B2가 '반례'(최소 n*)이면  전수의 최소 반례가 n*와 같아야 한다(n* <= 상한일 때).  다르면 대표 집합의 빈
틈이다."""
import json, multiprocessing as mp, os, sys, warnings
warnings.filterwarnings("ignore")
import oracle_close as oc, targets as T
from oracle_experiment import REL, KIND

BOUND = dict(rk4=60, ssprk3=60, ab3=60, fem=60, fvm=60, fdm2d=13)


def brute(rec):
    key, src = rec["key"], rec["src"]; name = T.ALL[key][0]
    try:
        fn, g = oc.exact_version(src, name, {})
    except Exception:
        return None
    first = None
    try:
        if key in REL:
            nmin = 3 if key == "ab3" else 1
            for n in range(nmin, BOUND[key] + 1):
```

```
                ok, info = oc.gf_time_run(fn, g, REL[key], n, 3, 99991 + n)
                if ok is None: return None
                if not ok: first = n; break
        elif key == "fdm2d":
            for s_ in range(4, 2 * BOUND[key] + 1):
                for nx in range(2, s_ - 1):
                    ny = s_ - nx
                    if ny < 2 or nx > BOUND[key] or ny > BOUND[key]: continue
                    ok, info = oc.gf_space2d_run(fn, g, nx, ny, 77 + 100 * nx + ny)
                    if ok is None: return None
                    if not ok: first = (nx, ny); break
                if first: break
        else:
            for N in range(2, BOUND[key] + 1):
                ok, info = oc.gf_space_run(fn, g, KIND[key], N, 55 + N, uniform=key == "fvm")
                if ok is None: return None
                if not ok: first = N; break
    except Exception as e:
        first = f"err {type(e).__name__}"
    return dict(key=key, kind=rec["kind"], B2=rec["B2"][0], B2info=rec["B2"][1], brute=first)


def main():
    R = json.load(open("oracle_results.json"))
    R = [r for r in R if r["B2"][0] in ("증명", "반례") and "시간 초과" not in str(r["B2"][1]) and "조건부:AB3" not in r["kind"]]
    with mp.Pool(os.cpu_count()) as p: out = [o for o in p.map(brute, R, chunksize=4) if o]
    bad = []
    for o in out:
        if o["B2"] == "증명" and o["brute"] is not None: bad.append(o)
        if o["B2"] == "반례" and o["brute"] is None:
            # B2의 최소 반례가 전수 상한 밖이면 일치
            import ast
            try: info = ast.literal_eval(o["B2info"]) if o["B2info"].startswith("{") else {}
            except Exception: info = {}
            nstar = info.get("n", info.get("N"))
            if nstar is None or (isinstance(nstar, int) and nstar <= BOUND[o["key"]]) or (isinstance(nstar, tuple) and max(nstar) <= BOUND[o["key"]]): bad.append(o)
        if o["B2"] == "반례" and o["brute"] is not None and not str(o["brute"]).startswith("err"):
            import ast
            try: info = ast.literal_eval(o["B2info"]) if o["B2info"].startswith("{") else {}
            except Exception: info = {}
            nstar = info.get("n", info.get("N"))
            sz = (lambda v: sum(v) if isinstance(v, tuple) else v)
            if nstar is not None and sz(o["brute"]) < sz(nstar): o["why"] = "전수가 더 작은 반례"; bad.append(o)
    json.dump(dict(n=len(out), bad=bad), open("brute_results.json", "w"), ensure_ascii=False, indent=0, default=str)
    print(f"교차 확인 {len(out)}개, 불일치 {len(bad)}개")
    for b in bad[:20]: print(b)


if __name__ == "__main__":
    main()
```

작동 확인 branch_test.py — 20.7절: 적응형 BS23의 결정 열거

```
"""값 분기 대상(적응형 BS23)을 세 오라클로 판정한다(20.5절)."""
import math, time, warnings, numpy as np
warnings.filterwarnings("ignore")
from fractions import Fraction as Fr
import oracle_close as oc, targets as T, oracle_experiment as E
A = [[0, 0, 0], [Fr(1, 2), 0, 0], [0, Fr(3, 4), 0]]; b = [Fr(2, 9), Fr(1, 3), Fr(4, 9)]


def check(out, f, args):
    ts, Y = out; y0, h0, Tend, tol = args
    if len(ts) != len(Y): return False, dict(why="t와 Y의 길이")
    if oc.GF.c(ts[0]).v != 0 or not oc.gf_eq(Y[0], y0)[0]: return False, dict(why="시작값")
    for k in range(len(Y) - 1):
        hk = oc.GF.c(ts[k + 1]) - oc.GF.c(ts[k])
        ok, c = oc.gf_eq(Y[k + 1], oc.rk_step(A, b, f, Y[k], hk))
        if not ok: return False, dict(step=k, why="받아들인 걸음의 관계식 위반")
    return True, {}


def inputs(rng):
    return (np.array([oc.gf_rand(rng) for _ in range(3)], dtype=object), oc.gf_rand(rng), oc.gf_rand(rng), oc.gf_rand(rng))
```

```

def b0(src):           # 제조해: tol을 줄이며 최종 오차가 tol을 따라 줄어드는가(적응형 코드의 표준 시험)
    fn = oc.float_version(src, "bs23_run", {}); errs = []
    for tol in (1e-4, 1e-5, 1e-6, 1e-7):
        ts, Y = fn(E.ode_f, E.Y0.copy(), 1e-3, 1.0, tol)
        if abs(ts[-1] - 1.0) > 1e-12 or not E.finite_list(Y): return True, "끝 시각·유한성"
        errs.append(float(np.max(np.abs(np.asarray(Y[-1]) - E.EX))))
    slope = math.log10(errs[0] / errs[-1]) / 3          # tol 10배당 오차 감소(자릿수)
    return (slope < 0.5 or errs[-1] > 1e-4, f"tol 10배당 오차 {slope:.2f}자리, 최종 {errs[-1]:.1e}")


def b1(src):           # 문서의 실행 검사: 사용자의 한 실행(h0 = 1e-3, tol = 1e-6)에서 받아들인 걸음마다 관계식
    from verify import check_rk
    fn = oc.float_version(src, "bs23_run", {}); S = E.sysS()
    ts, Y = fn(S.F, S.u0.copy(), 1e-3, 2.0, 1e-6); Y = np.array([np.asarray(v, float) for v in Y]); ts = np.array(ts, float)
    r = check_rk(S, ts, Y, [[float(a) for a in row] for row in A], [float(x) for x in b])
    nrej = None
    return (isinstance(r, tuple), (f"걸음 {r[0]}" if isinstance(r, tuple) else f"통과({len(ts) - 1}걸음)"))


def b2(src, D=12):
    return oc.close_branching(src, "bs23_run", {}, check, d=3, D=D, inputs=inputs)


if __name__ == "__main__":
    cases = [("BS23 원본", T.BS23)] + list(T.BRANCH_BUGS.items())
    for lab, src in cases:
        t0 = time.time()
        r0 = E.limited(lambda: b0(src), 30); r0 = (True, "시간 초과(끝나지 않음)") if r0 == ("timeout",) else r0
        r1 = E.limited(lambda: b1(src), 30); r1 = (True, "시간 초과(끝나지 않음)") if r1 == ("timeout",) else r1
        r2 = b2(src)
        print(f"{lab}\n  B0 {r0}\n  B1 {r1}\n  B2 {r2['verdict']} 경로 {r2.get('paths')} 잘림 {r2.get('truncated')} 고정 비교 {r2.get('fixed')} {r2.get('info', '')}  ({time.time() - t0:.1f}s)", flush=True)


def table():
    """20.7절의 표 줄(make_part5가 읽는다)."""
    import json
    rows = []
    for lab, src in [("BS23 원본", T.BS23)] + list(T.BRANCH_BUGS.items()):
        r0 = E.limited(lambda: b0(src), 30); r0 = (True, "끝나지 않음") if r0 == ("timeout",) else r0
        r1 = E.limited(lambda: b1(src), 30); r1 = (True, "끝나지 않음") if r1 == ("timeout",) else r1
        r2 = b2(src)
        m = lambda r: ("잡음: " if r[0] else "통과: ") + r[1][:26]
        b2s = f"{r2['verdict']}: 경로 {r2.get('paths')}개" + (f", 잘림 {r2.get('truncated')}" if r2['verdict'] == "증명" else f", {str(r2.get('info', {}).get('why', ''))[:20]}")
        rows.append(f"{lab} | {m(r0)} | {m(r1)} | {b2s}")
    return "\n".join(rows)
```

작동 확인 lorenz_test.py — 20.8절: 혼돈계에서 정확도 층

```

"""정확도 층의 한계: 혼돈계(로렌츠)에서 문서의 verify가 증명하는 시간 구간(21절)."""
import sys, time, numpy as np, warnings
warnings.filterwarnings("ignore")
from semidisc import Sys
from verify import verify, library
from schemes_lib import rk4
RK4c = library()["RK4"]
for T in (float(a) for a in sys.argv[1:]):
    S = Sys(dict(m=3, N=1, L=np.zeros((1, 1)), w=np.ones(1), D=[0.0, 0.0, 0.0], T=T, u0=[1.0, 1.0, 1.0],
                 R=lambda s: [10 * (s[1] - s[0]), s[0] * (28 - s[2]) - s[1], s[0] * s[1] - 8 * s[2] / 3]))
    t0 = time.time()
    rep = verify(S, step=rk4, N=int(400 * T), claim=RK4c, claim_name="RK4", eps=1e-3, name="로렌츠")
    print(f"T={T}: 판정 {rep['판정']['status']} | {rep['판정']['text'][:150]} | 언제: {rep.get('언제', {}).get('text', '')[:150]} ({time.time() - t0:.0f}s)", flush=True)
```

작동 확인 domains.py — 21절: 다른 분야의 오라클 문제

```
"""domains: 20절의 닫힘 장치(정확 산술 재실행 · 정수 추적 · 결정 열거)를 수치해석 밖의 오라클 문제에 그대로 쓴다(21절).
분야마다 (대상 코드, 의도한 명세, 입력 생성기)를 두고, 대상과 그 변이체를 판정한다.  되는지 안 되는지는 실험 결과로만
적는다.  기준선 BR은 그 분야의 흔한 시험(부동소수·실제 해시 등 한 번의 무작위 입력 차분 시험)이다.

모드
  gf      GF(p) 무작위 값 + 정수 추적(모든 값·모든 크기, 비아핀 술어면 상한까지 전수)
  int     파이썬 정수 그대로(이미 정확) + 정수 추적
  branch  값 분기를 결정 열거(모든 경로) + GF, 명세는 경로의 결정 기록을 쓸 수 있다
  perm    비교만 쓰는 코드: 크기 n의 모든 순서형(중복 포함)을 열거
  q       유리수(순서가 있는 정확 산술)로 표본 입력만 — 모든 입력은 닫지 못함(대조용)
"""
import cmath, hashlib, itertools, json, math, os, random, sys, time, warnings
import multiprocessing as mp
from fractions import Fraction as Fr
import numpy as np
warnings.filterwarnings("ignore")
import oracle_close as oc
from oracle_close import GF, Tracer, TInt, ValueBranch, gf_rand
import mutate

NTT_P = 998244353; NTT_G = 3; NTT_M = 1 << 23
SECP_P = 2 ** 256 - 2 ** 32 - 977
SECP_N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
SECP_G = (0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
          0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8)


# ---------------------------------------------------------------- 모든 크기 닫기(공통)
def close_sized(run, nmin, seeds=(1, 2), probe=None, min_bound=0):
    """run(n, seed, tracer) -> (ok, info).  대표 크기(20.3절)로 판정, 비아핀 술어면 상한까지 전수."""
    probe = probe if probe is not None else nmin + 4
    tr = Tracer(); ok, info = run(probe, seeds[0], tr)
    if ok is None: return dict(verdict="범위 밖", info=info)
    reps, meta = oc.representatives(tr.ev, nmin, extra=[probe]); seen = set(); fails = []
    while True:
        new = [r for r in reps if r not in seen]
        if not new: break
        for r in new:
            seen.add(r); tr2 = Tracer()
            for sd in seeds:
                ok, info = run(r, sd * 7919 + r, tr2)
                if ok is None: return dict(verdict="범위 밖", info=info, meta=meta)
                if not ok: fails.append((r, info)); break
            tr.ev += tr2.ev
        reps2, meta = oc.representatives(tr.ev, nmin); reps = sorted(set(reps) | set(reps2))
    if fails:
        r0 = min(f_[0] for f_ in fails)
        for r in range(nmin, r0):
            ok, info = run(r, 31 * r + 3, None)
            if ok is False: return dict(verdict="반례", n=r, info=info)
        return dict(verdict="반례", n=r0, info=[f_[1] for f_ in fails if f_[0] == r0][0])
    if meta["flags"] or min_bound:
        bound = max(max(seen), min_bound)
        for r in range(nmin, bound + 1):
            if r in seen: continue
            ok, info = run(r, 17 * r + 1, None)
            if ok is None: return dict(verdict="범위 밖", info=info)
            if ok is False: return dict(verdict="반례", n=r, info=info)
        return dict(verdict="유한 증명" if meta["flags"] else "증명", bound=bound, meta=meta)
    return dict(verdict="증명", reps=len(seen), meta=meta)


def gf_runner(fn, g, make, spec, field=None):
    def run(n, seed, tr):
        rng = random.Random(seed); tr = tr or Tracer()
        ctx = oc.field(field) if field else _null()
        with ctx:
            args = make(rng, n); old = oc._patched(g, tr)
            try:
                out = fn(*args)
            except ValueBranch as e:
                return None, dict(n=n, why=f"범위 밖: {e}")
            except Exception as e:
                return False, dict(n=n, why=f"실행 실패: {type(e).__name__}: {str(e)[:60]}")
            finally:
                oc._restore(g, old)
```

```
            try:
                ok, why = spec(args, out)
            except ValueBranch as e:
                return None, dict(n=n, why=f"명세가 범위 밖: {e}")
            except Exception as e:
                return False, dict(n=n, why=f"출력 형식: {type(e).__name__}: {str(e)[:60]}")
        return ok, dict(n=n, why=why)
    return run


class _null:
    def __enter__(s): return s
    def __exit__(s, *a): return False


def branch_close(fn, make, spec, D=14, seed=1, field=None):
    """값 분기의 모든 결정 경로(D 이내)에서 spec(args, out, log).  같음 갈래는 경로 조건을 입력에 대입해 판정(21.3절)."""
    stack = [([], {})]; paths = trunc = 0; unres = None
    ctx = oc.field(field) if field else _null()
    with ctx:
        oc._LIN[0] = True
        try:
            while stack:
                pre, pins = stack.pop()
                rng = random.Random(seed * 7777 + len(pre) * 31 + sum(b << i for i, b in enumerate(pre)))
                args = make(rng); oc._BR[0] = sch = oc.Sched(pre, rng, pins); oc.bind_inputs(args, pins, sch)
                try:
                    out = fn(*args)
                except oc.NeedMore:
                    if len(pre) >= D: trunc += 1
                    else: stack += [(pre + [False], pins), (pre + [True], pins)]
                    continue
                except oc.Pin as e:
                    stack.append((pre[:-1], oc.add_pin(pins, e))); continue
                except ValueBranch as e:
                    if "같음 갈래" in str(e): unres = unres or str(e); continue
                    return dict(verdict="범위 밖", info=str(e))
                except Exception as e:
                    return dict(verdict="반례", info=f"경로 {sch.log} {oc.pins_str(pins)}: 실행 실패 {type(e).__name__}")
                finally:
                    oc._BR[0] = None
                oc._BR[0] = sch
                try:
                    ok, why = spec(args, out, sch.log)
                except ValueBranch as e:
                    return dict(verdict="범위 밖", info=f"명세: {e}")
                finally:
                    oc._BR[0] = None
                paths += 1
                if not ok: return dict(verdict="반례", info=f"경로 {sch.log} {oc.pins_str(pins)}: {why}")
        finally:
            oc._BR[0] = None; oc._LIN[0] = False
    if unres: return dict(verdict="범위 밖", info=unres)
    return dict(verdict="증명" if trunc == 0 else f"유한 증명", paths=paths, truncated=trunc, D=D)


def rmat(rng, n, m=None): return [[gf_rand(rng) for _ in range(m or n)] for _ in range(n)]
def rvec(rng, n): return [gf_rand(rng) for _ in range(n)]
def v(x): return GF.c(x).v
def veq(a, b): return len(a) == len(b) and all(v(x) == v(y) for x, y in zip(a, b))
def gmm(A, B):
    return [[sum((A[i][k] * B[k][j] for k in range(len(B))), GF(0)) for j in range(len(B[0]))] for i in range(len(A))]


# ---------------------------------------------------------------- 분야
D = {}

D["matmul"] = dict(area="선형대수", label="행렬곱(삼중 루프)", mode="gf", nmin=1, fname="matmul", src='''
def matmul(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
```

```
            s = 0
            for k in range(n):
                s = s + A[i][k] * B[k][j]
            C[i][j] = s
    return C
''', make=lambda rng, n: (rmat(rng, n), rmat(rng, n)),
    spec=lambda a, out: (all(veq(r1, r2) for r1, r2 in zip(out, gmm(a[0], a[1]))) and len(out) == len(a[0]), "C = AB"),
    fmake=lambda rng: (rng.standard_normal((6, 6)).tolist(), rng.standard_normal((6, 6)).tolist()),
    fspec=lambda a, out: np.allclose(out, np.array(a[0]) @ np.array(a[1]), rtol=1e-9, atol=1e-12))


def lu_spec(a, out):
    A = a[0]; L, U = out; n = len(A)
    for i in range(n):
        if v(L[i][i]) != 1: return False, "L 대각 != 1"
        for j in range(n):
            if j > i and v(L[i][j]) != 0: return False, "L 위쪽 != 0"
            if j < i and v(U[i][j]) != 0: return False, "U 아래쪽 != 0"
    return all(veq(r1, r2) for r1, r2 in zip(gmm(L, U), A)), "LU = A"


def lu_fspec(a, out):
    A = np.array(a[0]); L, U = np.array(out[0], float), np.array(out[1], float)
    return np.allclose(L @ U, A, atol=1e-9) and np.allclose(np.triu(L, 1), 0) and np.allclose(np.tril(U, -1), 0) and np.allclose(np.diag(L), 1)


D["lu"] = dict(area="선형대수", label="LU 분해(두리틀, 피벗 없음)", mode="gf", nmin=1, fname="lu", src='
''
def lu(A):
    n = len(A)
    L = [[0] * n for _ in range(n)]
    U = [[0] * n for _ in range(n)]
    for i in range(n):
        L[i][i] = 1
        for j in range(i, n):
            s = 0
            for k in range(i):
                s = s + L[i][k] * U[k][j]
            U[i][j] = A[i][j] - s
        for j in range(i + 1, n):
            s = 0
            for k in range(i):
                s = s + L[j][k] * U[k][i]
            L[j][i] = (A[j][i] - s) / U[i][i]
    return L, U
''', make=lambda rng, n: (rmat(rng, n),), spec=lu_spec,
    fmake=lambda rng: ((rng.standard_normal((6, 6)) + 8 * np.eye(6)).tolist(),), fspec=lu_fspec)


def sym(rng, n):
    A = rmat(rng, n)
    return [[A[i][j] if i <= j else A[j][i] for j in range(n)] for i in range(n)]


D["cg"] = dict(area="선형대수", label="켤레기울기법(n번 반복)", mode="gf", nmin=1, fname="cg", src='''
def cg(A, b):
    n = len(b)
    x = [0] * n
    r = list(b)
    p = list(b)
    rr = 0
    for i in range(n):
        rr = rr + r[i] * r[i]
    for it in range(n):
        Ap = [0] * n
        for i in range(n):
            for j in range(n):
                Ap[i] = Ap[i] + A[i][j] * p[j]
        pAp = 0
        for i in range(n):
            pAp = pAp + p[i] * Ap[i]
        alpha = rr / pAp
        for i in range(n):
            x[i] = x[i] + alpha * p[i]
            r[i] = r[i] - alpha * Ap[i]
        rr_new = 0
        for i in range(n):
            rr_new = rr_new + r[i] * r[i]
```

```
        beta = rr_new / rr
        for i in range(n):
            p[i] = r[i] + beta * p[i]
        rr = rr_new
    return x
''', make=lambda rng, n: (sym(rng, n), rvec(rng, n)),
    spec=lambda a, out: (veq([sum((a[0][i][j] * out[j] for j in range(len(out))), GF(0)) for i in range(len(out))], a[1]), "A x = b"),
    fmake=lambda rng: (lambda M: ((M @ M.T + 6 * np.eye(6)).tolist(), rng.standard_normal(6).tolist()))(rng.standard_normal((6, 6))),
    fspec=lambda a, out: np.linalg.norm(np.array(a[0]) @ np.array(out, float) - np.array(a[1])) <= 1e-8 * np.linalg.norm(a[1]))


# FFT: 코드의 cmath.exp(-2j*pi*k/n)를 NTT 체의 단위근으로 가로챈다(초월 상수의 대수적 특수값)
class _Ang:
    """c·π·i (c 유리수)"""
    def __init__(s, c): s.c = c
    def __mul__(s, o):
        if isinstance(o, complex): return _Ang(s.c * oc.rat(o.imag)) if o.real == 0 else NotImplemented
        if isinstance(o, (int, Fr)): return _Ang(s.c * Fr(int(o)) if isinstance(o, int) else s.c * o)
        return NotImplemented
    __rmul__ = __mul__
    def __truediv__(s, o): return _Ang(s.c / Fr(int(o)))
    def __neg__(s): return _Ang(-s.c)


class _PI:
    def __rmul__(s, o):
        if isinstance(o, complex) and o.real == 0: return _Ang(oc.rat(o.imag))
        return NotImplemented
    __mul__ = __rmul__


class _CM:
    pi = _PI()
    @staticmethod
    def exp(a):
        if not isinstance(a, _Ang): raise ValueBranch("초월 함수 exp의 비특수값")
        e = a.c / 2 * NTT_M                       # exp(c·π·i) = ω^(c/2 · M),  ω = M제곱 단위근
        if e.denominator != 1: raise ValueBranch("체에 없는 단위근")
        w = pow(NTT_G, (NTT_P - 1) // NTT_M, NTT_P)
        return GF(pow(w, int(e) % NTT_M, NTT_P))


def dft_spec(a, out):
    x = a[0]; n = len(x); w = pow(NTT_G, (NTT_P - 1) // NTT_M, NTT_P)
    wn = pow(w, (NTT_M - NTT_M // n) % NTT_M, NTT_P)      # exp(-2πi/n)
    X = [sum((x[j] * GF(pow(wn, (j * k) % n, NTT_P)) for j in range(n)), GF(0)) for k in range(n)]
    return veq(out, X), "DFT"


D["fft"] = dict(area="신호처리", label="FFT(재귀 2진, cmath 단위근)", mode="gf", nmin=0, fname="fft", field=NTT_P,
                env_exact=dict(cmath=_CM), env_float=dict(cmath=cmath), min_bound=8, src='''
def fft(x):
    n = len(x)
    if n == 1:
        return [x[0]]
    even = fft(x[0::2])
    odd = fft(x[1::2])
    out = [0] * n
    for k in range(n // 2):
        w = cmath.exp(-2j * cmath.pi * k / n)
        out[k] = even[k] + w * odd[k]
        out[k + n // 2] = even[k] - w * odd[k]
    return out
''', make=lambda rng, r: (rvec(rng, 1 << r),), spec=dft_spec,
    fmake=lambda rng: (list(rng.standard_normal(16) + 0j),), fspec=lambda a, out: np.allclose(out, np.fft.fft(a[0]), atol=1e-9))


def conv_spec(a, out):
    x, y = a; n = len(x); r = [GF(0)] * (2 * n - 1)
    for i in range(n):
        for j in range(n): r[i + j] = r[i + j] + x[i] * y[j]
    return veq(out, r), "합성곱"


D["conv"] = dict(area="신호처리", label="직접 합성곱", mode="gf", nmin=1, fname="conv", src='''
```

```
def conv(a, b):
    n = len(a)
    out = [0] * (2 * n - 1)
    for i in range(n):
        for j in range(n):
            out[i + j] = out[i + j] + a[i] * b[j]
    return out
''', make=lambda rng, n: (rvec(rng, n), rvec(rng, n)), spec=conv_spec,
    fmake=lambda rng: (rng.standard_normal(7).tolist(), rng.standard_normal(7).tolist()), fspec=lambda a
, out: np.allclose(out, np.convolve(a[0], a[1])))


def pdiv_spec(a, out):
    A, B = a; q, r = out
    if len(r) != len(B) - 1 or len(q) != len(A) - len(B) + 1: return False, "길이"
    prod = [GF(0)] * (len(q) + len(B) - 1)
    for i in range(len(q)):
        for j in range(len(B)): prod[i + j] = prod[i + j] + q[i] * B[j]
    for i in range(len(r)): prod[i] = prod[i] + r[i]
    return veq(prod, A), "q b + r = a"


D["polydiv"] = dict(area="기호·대수", label="다항식 나눗셈", mode="gf", nmin=1, fname="poly_divmod", src
='''
def poly_divmod(a, b):
    r = list(a)
    q = [0] * (len(a) - len(b) + 1)
    for i in range(len(a) - len(b), -1, -1):
        c = r[i + len(b) - 1] / b[len(b) - 1]
        q[i] = c
        for j in range(len(b)):
            r[i + j] = r[i + j] - c * b[j]
    return q, r[:len(b) - 1]
''', make=lambda rng, n: (rvec(rng, n + 3), rvec(rng, n)), spec=pdiv_spec,
    fmake=lambda rng: (rng.standard_normal(9).tolist(), rng.standard_normal(4).tolist()),
    fspec=lambda a, out: np.allclose(np.polynomial.polynomial.polyadd(np.polynomial.polynomial.polymul(o
ut[0], a[1]), out[1]), a[0], atol=1e-8))


def gdet(A):
    M = [list(r) for r in A]; n = len(M); d = GF(1)
    for k in range(n):
        piv = next((i for i in range(k, n) if v(M[i][k]) != 0), None)
        if piv is None: return GF(0)
        if piv != k: M[k], M[piv] = M[piv], M[k]; d = -d
        d = d * M[k][k]
        for i in range(k + 1, n):
            f = M[i][k] / M[k][k]
            for j in range(k, n): M[i][j] = M[i][j] - f * M[k][j]
    return d


D["bareiss"] = dict(area="기호·대수", label="행렬식(바라이스)", mode="gf", nmin=1, fname="det_bareiss",
src='''
def det_bareiss(A):
    n = len(A)
    M = [list(row) for row in A]
    prev = 1
    for k in range(n - 1):
        for i in range(k + 1, n):
            for j in range(k + 1, n):
                M[i][j] = (M[i][j] * M[k][k] - M[i][k] * M[k][j]) / prev
        prev = M[k][k]
    return M[n - 1][n - 1]
''', make=lambda rng, n: (rmat(rng, n),), spec=lambda a, out: (v(out) == v(gdet(a[0])), "det"),
    fmake=lambda rng: (rng.standard_normal((5, 5)).tolist(),), fspec=lambda a, out: abs(out - np.linalg.
det(a[0])) < 1e-8 * max(1, abs(np.linalg.det(a[0]))))


def lsq_spec(a, out):
    A, b = a; m, n = len(A), len(A[0])
    if len(out) != n: return False, f"출력 길이 {len(out)} != {n}"
    res = [sum((A[k][j] * out[j] for j in range(n)), GF(0)) - b[k] for k in range(m)]
    return veq([sum((A[k][i] * res[k] for k in range(m)), GF(0)) for i in range(n)], [GF(0)] * n), "Aᵀ(A
x - b) = 0"


LSQ_SRC = '''
def lstsq(A, b):
    m = len(A)
```

```
    n = len(A[0])
    G = [[0] * n for _ in range(n)]
    c = [0] * n
    for i in range(n):
        for j in range(n):
            for k in range(m):
                G[i][j] = G[i][j] + A[k][i] * A[k][j]
        for k in range(m):
            c[i] = c[i] + A[k][i] * b[k]
    for i in range(n):
        for r in range(i + 1, n):
            f = G[r][i] / G[i][i]
            for j in range(i, n):
                G[r][j] = G[r][j] - f * G[i][j]
            c[r] = c[r] - f * c[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        s = c[i]
        for j in range(i + 1, n):
            s = s - G[i][j] * x[j]
        x[i] = s / G[i][i]
    return x
'''
D["lstsq"] = dict(area="최적화", label="최소제곱(정규방정식)", mode="gf", nmin=1, fname="lstsq", src=LSQ
_SRC,
                  make=lambda rng, n: (rmat(rng, n + 2, n), rvec(rng, n + 2)), spec=lsq_spec,
                  fmake=lambda rng: (rng.standard_normal((8, 5)).tolist(), rng.standard_normal(8).tolist
()),
                  fspec=lambda a, out: np.allclose(out, np.linalg.lstsq(np.array(a[0]), np.array(a[1]),
rcond=None)[0], atol=1e-8))

D["gd"] = dict(area="최적화", label="경사하강(최소제곱, 고정 반복)", mode="gf", nmin=1, fname="gd_lstsq"
, src='''
def gd_lstsq(A, b, steps, lr):
    n = len(A[0])
    x = [0] * n
    for it in range(steps):
        r = [sum(A[k][j] * x[j] for j in range(n)) - b[k] for k in range(len(A))]
        g = [sum(A[k][i] * r[k] for k in range(len(A))) for i in range(n)]
        x = [x[i] - lr * g[i] for i in range(n)]
    return x
''', make=lambda rng, n: (rmat(rng, n + 2, n), rvec(rng, n + 2), 200, gf_rand(rng)), spec=lambda a, out:
 lsq_spec(a[:2], out),
    fmake=lambda rng: (rng.standard_normal((8, 4)).tolist(), rng.standard_normal(8).tolist(), 3000, 0.02
),
    fspec=lambda a, out: np.allclose(out, np.linalg.lstsq(np.array(a[0]), np.array(a[1]), rcond=None)[0]
, atol=1e-6))

D["power"] = dict(area="고유값", label="거듭제곱법(고정 반복)", mode="gf", nmin=2, fname="power_iter", s
rc='''
def power_iter(A, steps):
    n = len(A)
    x = [1] * n
    lam = 0
    for it in range(steps):
        y = [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
        lam = sum(x[i] * y[i] for i in range(n)) / sum(x[i] * x[i] for i in range(n))
        s = y[0]
        x = [y[i] / s for i in range(n)]
    return lam, x
''', make=lambda rng, n: (sym(rng, n), 60),
    spec=lambda a, out: (veq([sum((a[0][i][j] * out[1][j] for j in range(len(out[1]))), GF(0)) for i in
range(len(out[1]))], [out[0] * t for t in out[1]]), "A v = λ v"),
    fmake=lambda rng: (lambda M: ((M + M.T + np.diag([9.0, 1, 0.5, 0.2, 0])).tolist(), 400))(rng.standar
d_normal((5, 5)) * 0.1),
    fspec=lambda a, out: abs(out[0] - max(np.linalg.eigvalsh(a[0]), key=abs)) < 1e-8)

D["segint"] = dict(area="계산기하", label="선분 교차 판정(방향 부호)", mode="gf", nmin=1, fname="interse
cts", src='''
def intersects(p1, p2, p3, p4):
    d1 = (p4[0] - p3[0]) * (p1[1] - p3[1]) - (p4[1] - p3[1]) * (p1[0] - p3[0])
    d2 = (p4[0] - p3[0]) * (p2[1] - p3[1]) - (p4[1] - p3[1]) * (p2[0] - p3[0])
    d3 = (p2[0] - p1[0]) * (p3[1] - p1[1]) - (p2[1] - p1[1]) * (p3[0] - p1[0])
    d4 = (p2[0] - p1[0]) * (p4[1] - p1[1]) - (p2[1] - p1[1]) * (p4[0] - p1[0])
    return ((d1 > 0) != (d2 > 0)) and ((d3 > 0) != (d4 > 0))
''', make=lambda rng, n: tuple(rvec(rng, 2) for _ in range(4)), spec=lambda a, out: (True, ""),
    fmake=lambda rng: tuple(rng.uniform(0, 1, 2).tolist() for _ in range(4)), fspec=None)


def seg_ref(p1, p2, p3, p4):
```

```
    M = np.array([[p2[0] - p1[0], p3[0] - p4[0]], [p2[1] - p1[1], p3[1] - p4[1]]])
    if abs(np.linalg.det(M)) < 1e-14: return False
    s, t = np.linalg.solve(M, [p3[0] - p1[0], p3[1] - p1[1]]); return 0 < s < 1 and 0 < t < 1


D["segint"]["fspec"] = lambda a, out: out == seg_ref(*a)

D["isort"] = dict(area="정렬·탐색", label="삽입 정렬", mode="perm", nmin=0, fname="insertion_sort", bound=6, src='''
def insertion_sort(a):
    a = list(a)
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j = j - 1
        a[j + 1] = key
    return a
''', fmake=lambda rng: (rng.integers(0, 100, 20).tolist(),), fspec=lambda a, out: out == sorted(a[0]))

D["bsearch"] = dict(area="정렬·탐색", label="이분 탐색(왼쪽 끝)", mode="perm", nmin=0, fname="lower_bound", bound=7, src='''
def lower_bound(a, x):
    lo = 0
    hi = len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
''', fmake=lambda rng: (sorted(rng.integers(0, 50, 30).tolist()), int(rng.integers(0, 50))),
    fspec=lambda a, out: out == next((i for i, t in enumerate(a[0]) if t >= a[1]), len(a[0])))


def sin_poly(x):
    """주장한 다항식: sum_{k=0..7} (-1)^k x^(2k+1) / (2k+1)!"""
    s = GF(0); t = x
    for k in range(8):
        s = s + t * GF.c(Fr((-1) ** k, math.factorial(2 * k + 1))); t = t * x * x
    return s


SIN_SRC = '''
def sin_taylor(x):
    term = x
    s = x
    for k in range(1, 8):
        term = -term * x * x / ((2 * k) * (2 * k + 1))
        s = s + term
    return s
'''
D["sin_true"] = dict(area="초월함수", label="sin 근사(명세: 참 sin)", mode="gf", nmin=1, fname="sin_taylor", src=SIN_SRC,
                     make=lambda rng, n: (gf_rand(rng),), spec=lambda a, out: (v(out) == v(GF.c(math.sin(float(a[0]))))), "sin"),
                     fmake=lambda rng: (float(rng.uniform(-1, 1)),), fspec=lambda a, out: abs(out - math.sin(a[0])) < 1e-12)
D["sin_poly"] = dict(D["sin_true"], label="sin 근사(명세: 주장한 15차 다항식)", spec=lambda a, out: (v(out) == v(sin_poly(a[0])), "다항식"))

D["kahan"] = dict(area="반올림 의존", label="카한 합", mode="gf", nmin=1, fname="kahan_sum", src='''
def kahan_sum(xs):
    s = 0
    c = 0
    for x in xs:
        y = x - c
        t = s + y
        c = (t - s) - y
        s = t
    return s
''', make=lambda rng, n: (rvec(rng, n),), spec=lambda a, out: (v(out) == v(sum(a[0], GF(0))), "Σx"),
    fmake=lambda rng: ([1.0] + [1e-16] * 10000 + [-1.0],), fspec=lambda a, out: abs(out - 1e-12) <= 2 * 2 ** -53 * 2 + 1e-24)


D["lcg"] = dict(area="난수", label="선형 합동 난수열", mode="int", nmin=0, fname="lcg", src='''
def lcg(seed, n):
```

```
    out = []
    x = seed
    for i in range(n):
        x = (1103515245 * x + 12345) % 2147483648
        out.append(x)
    return out
''', fmake=lambda rng: (12345, 50), fspec=None)


def lcg_ref(seed, n):
    out, x = [], seed
    for _ in range(int(n)): x = (1103515245 * x + 12345) % 2147483648; out.append(x)
    return out


D["lcg"]["fspec"] = lambda a, out: out == lcg_ref(*a)

D["mcpi"] = dict(area="몬테카를로", label="몬테카를로 π(명세: ≈ π)", mode="gf", nmin=1, fname="mc_pi", s
rc='''
def mc_pi(points):
    hits = 0
    for p in points:
        if p[0] * p[0] + p[1] * p[1] <= 1:
            hits = hits + 1
    return 4 * hits / len(points)
''', make=lambda rng, n: ([rvec(rng, 2) for _ in range(n)],), spec=lambda a, out: (True, ""),
    fmake=lambda rng: (rng.uniform(0, 1, (20000, 2)).tolist(),), fspec=lambda a, out: abs(out - math.pi)
 < 0.05)

D["rpn"] = dict(area="인터프리터", label="후위 표기 계산기", mode="prog", nmin=1, fname="eval_rpn", src=
'''
def eval_rpn(tokens, env):
    st = []
    for t in tokens:
        if t == "+":
            b = st.pop()
            a = st.pop()
            st.append(a + b)
        elif t == "-":
            b = st.pop()
            a = st.pop()
            st.append(a - b)
        elif t == "*":
            b = st.pop()
            a = st.pop()
            st.append(a * b)
        elif t == "/":
            b = st.pop()
            a = st.pop()
            st.append(a / b)
        else:
            st.append(env[t])
    return st[-1]
''', fmake=None, fspec=None)


def mlp_make(rng):
    H, I, O = 4, 3, 2
    return (rvec(rng, I), rmat(rng, H, I), rvec(rng, H), rmat(rng, O, H), rvec(rng, O))


def mlp_spec(a, out, log):
    x, W1, b1, W2, b2 = a
    if len(log) != len(W1): return False, f"결정 수 {len(log)} != 은닉 {len(W1)}"
    h = []
    for i in range(len(W1)):
        s = b1[i] + sum((W1[i][j] * x[j] for j in range(len(x))), GF(0))
        h.append(GF(0) if log[i] else s)       # max(s, 0): 0 > s 이 참이면 0
    y = [b2[i] + sum((W2[i][j] * h[j] for j in range(len(h))), GF(0)) for i in range(len(W2))]
    return veq(out, y), "경로마다 선형 조각"


D["mlp"] = dict(area="머신러닝", label="ReLU 신경망 추론(가중치 주어짐)", mode="branch", fname="mlp", sr
c='''
def mlp(x, W1, b1, W2, b2):
    h = []
    for i in range(len(W1)):
        s = b1[i]
        for j in range(len(x)):
            s = s + W1[i][j] * x[j]
```

```
        h.append(max(s, 0))
    out = []
    for i in range(len(W2)):
        s = b2[i]
        for j in range(len(h)):
            s = s + W2[i][j] * h[j]
        out.append(s)
    return out
''', make=mlp_make, spec=mlp_spec, D=8,
    fmake=lambda rng: (rng.standard_normal(3).tolist(), rng.standard_normal((4, 3)).tolist(), rng.standa
rd_normal(4).tolist(),
                       rng.standard_normal((2, 4)).tolist(), rng.standard_normal(2).tolist()),
    fspec=lambda a, out: np.allclose(out, np.array(a[3]) @ np.maximum(np.array(a[1]) @ np.array(a[0]) +
a[2], 0) + a[4]))

# ---------------------------------------------------------------- 블록체인
EC_SRC = '''
def ec_add(P1, P2):
    if P1 is None:
        return P2
    if P2 is None:
        return P1
    x1, y1 = P1
    x2, y2 = P2
    if x1 == x2 and (y1 + y2) % PP == 0:
        return None
    if x1 == x2:
        lam = 3 * x1 * x1 * pow(2 * y1, PP - 2, PP) % PP
    else:
        lam = (y2 - y1) * pow(x2 - x1, PP - 2, PP) % PP
    x3 = (lam * lam - x1 - x2) % PP
    y3 = (lam * (x1 - x3) - y1) % PP
    return (x3, y3)
'''


def ec_ref(P1, P2):
    """SEC 1의 아핀 군 법칙(대상과 독립: 사영 좌표로 계산해 아핀으로 되돌림)."""
    p = SECP_P
    if P1 is None: return P2
    if P2 is None: return P1
    X1, Y1, Z1 = P1[0], P1[1], 1; X2, Y2, Z2 = P2[0], P2[1], 1
    U1, U2, S1, S2 = X1 * Z2 % p, X2 * Z1 % p, Y1 * Z2 % p, Y2 * Z1 % p
    if U1 == U2:
        if (S1 + S2) % p == 0: return None
        W = 3 * X1 * X1 % p; S = Y1 * Z1 % p; B = X1 * Y1 * S % p; Hh = (W * W - 8 * B) % p
        X3 = 2 * Hh * S % p; Y3 = (W * (4 * B - Hh) - 8 * Y1 * Y1 * S * S) % p; Z3 = 8 * S ** 3 % p
    else:
        u = (S2 - S1) % p; w = (U2 - U1) % p; A = (u * u * Z1 * Z2 - w ** 3 - 2 * w * w * U1) % p
        X3 = w * A % p; Y3 = (u * (w * w * U1 - A) - w ** 3 * S1) % p; Z3 = w ** 3 * Z1 * Z2 % p
    zi = pow(Z3, p - 2, p); return (X3 * zi % p, Y3 * zi % p)


def rand_point(rng):
    p = SECP_P
    while True:
        x = rng.randrange(p); r = (x ** 3 + 7) % p; y = pow(r, (p + 1) // 4, p)
        if y * y % p == r: return (x, y)


BETA = next(b for b in (pow(g_, (SECP_P - 1) // 3, SECP_P) for g_ in range(2, 50)) if b != 1)   # 1의 원
시 세제곱근


def ec_make(rng, case):
    """코드의 같음 술어 (x1 == x2, y1 + y2 ≡ 0)의 참거짓 조합마다 그 층의 점을 만든다(21.3절).
    0 일반(F,F)  1 두 배(T,F)  2 역원(T,T)  3·4 무한원점  5·6 자기사상 쌍 (x, y), (βx, -y): (F,T) — 두
순서."""
    P1 = rand_point(rng)
    if case == 0: return (P1, rand_point(rng))
    if case == 1: return (P1, P1)
    if case == 2: return (P1, (P1[0], (-P1[1]) % SECP_P))
    if case == 3: return (None, P1)
    if case == 4: return (P1, None)
    Q = (BETA * P1[0] % SECP_P, (-P1[1]) % SECP_P)
    return (P1, Q) if case == 5 else (Q, P1)


D["ecadd"] = dict(area="블록체인", label="secp256k1 점 덧셈", mode="ec", fname="ec_add", src=EC_SRC, env
=dict(PP=SECP_P),
```

```
                    fmake=lambda rng: ec_make(random.Random(int(rng.integers(1 << 30))), 0), fspec=lambda a, out: out == ec_ref(*a))

ECMUL_SRC = '''
def ec_mul(k, P1):
    R = None
    Q = P1
    while k > 0:
        if k % 2 == 1:
            R = ec_add(R, Q)
        Q = ec_add(Q, Q)
        k = k // 2
    return R
'''
D["ecmul"] = dict(area="블록체인", label="secp256k1 스칼라 곱(두 배-더하기)", mode="ecmul", fname="ec_mul", src=ECMUL_SRC,
                    fmake=None, fspec=None)


class _Hash:
    """hashlib 자리의 무작위 신탁: sha256(b).digest()는 b마다 고정된 무작위 32바이트(해석하지 않은 함수)."""
    def __init__(s, rng): s.rng, s.tab = rng, {}
    def sha256(s, b=b""):
        hs = s
        class _H:
            def digest(_):
                b_ = bytes(b)
                if b_ not in hs.tab: hs.tab[b_] = bytes(hs.rng.randrange(256) for _ in range(32))
                return hs.tab[b_]
        return _H()


def merkle_ref(leaves, H):
    lv = [H.sha256(x).digest() for x in leaves]
    while len(lv) > 1:
        if len(lv) % 2: lv = lv + [lv[-1]]
        lv = [H.sha256(lv[i] + lv[i + 1]).digest() for i in range(0, len(lv), 2)]
    return lv[0]


D["merkle"] = dict(area="블록체인", label="머클 루트(비트코인식 홀수 복제)", mode="hash", nmin=1, fname="merkle_root", min_bound=40,
                   env_float=dict(hashlib=hashlib), src='''
def merkle_root(leaves):
    level = [hashlib.sha256(x).digest() for x in leaves]
    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1])
        nxt = []
        for i in range(0, len(level), 2):
            nxt.append(hashlib.sha256(level[i] + level[i + 1]).digest())
        level = nxt
    return level[0]
''', fmake=lambda rng: ([bytes([i]) * 8 for i in range(8)],), fspec=lambda a, out: out == merkle_ref(a[0], hashlib))


def ledger_make(rng):
    bal = {a: gf_rand(rng) for a in "ABCD"}; frm = rng.choice("ABCD"); to = rng.choice("ABCD")
    return (bal, frm, to, gf_rand(rng))


def ledger_spec(a, out, log):
    bal, frm, to, amt = a; nb, ok = out
    if set(nb) != set(bal): return False, "계정 집합"
    if v(sum(nb.values(), GF(0))) != v(sum(bal.values(), GF(0))): return False, "총량 보존 위반"
    if not ok:
        return all(v(nb[k]) == v(bal[k]) for k in bal), "거절이면 그대로"
    exp = dict(bal); exp[frm] = exp[frm] - amt; exp[to] = exp[to] + amt
    return all(v(nb[k]) == v(exp[k]) for k in bal), "이전"


D["ledger"] = dict(area="블록체인", label="잔액 이전(require 잔액 >= 금액)", mode="branch", fname="transfer", src='''
def transfer(bal, frm, to, amt):
    bal = dict(bal)
    if bal[frm] < amt:
        return bal, False
    bal[frm] = bal[frm] - amt
```

```
    bal[to] = bal[to] + amt
    return bal, True
''', make=ledger_make, spec=ledger_spec, D=4,
    fmake=lambda rng: ({"A": 100, "B": 5, "C": 0, "D": 7}, "A", "B", 100),
    fspec=lambda a, out: out[1] is True and out[0]["A"] == 0 and out[0]["B"] == 105)
# 기준선: 경계값(잔액 = 금액) 한 번의 시험 — 흔히 쓰는 경계 시험


# ---------------------------------------------------------------- 판정
def judge(key, src):
    d = D[key]; mode = d["mode"]
    if mode in ("gf",):
        fn, g = oc.exact_version(src, d["fname"], d.get("env_exact", d.get("env", {})))
        return close_sized(gf_runner(fn, g, d["make"], d["spec"], d.get("field")), d["nmin"], min_bound=
d.get("min_bound", 0))
    if mode == "int":
        fn = oc.float_version(src, d["fname"], {}); g = fn.__globals__
        def run(n, seed, tr):
            rng = random.Random(seed); s0 = rng.randrange(1 << 31); tr = tr or Tracer(); old = oc._patch
ed(g, tr)
            try: out = fn(s0, TInt(n, tr, {"n"}))
            except Exception as e: return False, dict(n=n, why=f"실행 실패 {type(e).__name__}")
            finally: oc._restore(g, old)
            return out == lcg_ref(s0, n), dict(n=n)
        return close_sized(run, d["nmin"])
    if mode == "perm":
        fn = oc.float_version(src, d["fname"], {}); g = fn.__globals__
        class Key:
            __slots__ = ("k",)
            def __init__(s, k): s.k = k
            def __lt__(s, o): return s.k < (o.k if isinstance(o, Key) else o)
            def __le__(s, o): return s.k <= (o.k if isinstance(o, Key) else o)
            def __gt__(s, o): return s.k > (o.k if isinstance(o, Key) else o)
            def __ge__(s, o): return s.k >= (o.k if isinstance(o, Key) else o)
            def __eq__(s, o): return s.k == (o.k if isinstance(o, Key) else o)
            __hash__ = None
        for n in range(d["nmin"], d["bound"] + 1):
            words = itertools.product(range(n), repeat=n) if key == "isort" else itertools.combinations_
with_replacement(range(n), n)
            for w in words:                                      # 크기 n의 모든 순서형(중복 포함; 탐색
은 정렬된 것만)
                try:
                    if key == "isort":
                        out = fn([Key(t) for t in w]); ok = [t.k for t in out] == sorted(w)
                    else:
                        ws = sorted(w)
                        for x in range(-1, n + 1):
                            out = fn([Key(t) for t in ws], Key(x)); ok = out == next((i for i, t in enum
erate(ws) if t >= x), n)
                            if not ok: break
                except Exception as e:
                    return dict(verdict="반례", n=n, info=f"{w}: 실행 실패 {type(e).__name__}")
                if not ok: return dict(verdict="반례", n=n, info=str(w))
        return dict(verdict="유한 증명", bound=d["bound"])
    if mode == "branch":
        fn, g = oc.exact_version(src, d["fname"], {})
        return branch_close(fn, d["make"], d["spec"], D=d.get("D", 12))
    if mode == "prog":
        return judge_rpn(src)
    if mode == "ec":
        fn = oc.float_version(src, d["fname"], d["env"])
        for case in range(7):
            for sd in range(6):
                a = ec_make(random.Random(1000 * case + sd), case)
                try: out = fn(*a)
                except Exception as e: return dict(verdict="반례", info=f"경우 {case}: 실행 실패 {type(e
).__name__}")
                if out != ec_ref(*a): return dict(verdict="반례", info=f"경우 {case}")
        return dict(verdict="층별 증명", info="같음 술어의 층 7가지 x 무작위 점(층마다 체 원소 식의 항등
, SZ)")
    if mode == "ecmul":
        return judge_ecmul(src)
    if mode == "hash":
        fn, g0 = oc.exact_version(src, d["fname"], {})
        def run(n, seed, tr):
            rng = random.Random(seed); H = _Hash(rng); g0["hashlib"] = H
            leaves = [bytes(rng.randrange(256) for _ in range(4)) for _ in range(n)]
            tr = tr or Tracer(); old = oc._patched(g0, tr)
            try: out = fn(leaves)
            except Exception as e: return False, dict(n=n, why=f"실행 실패 {type(e).__name__}")
```

```
            finally: oc._restore(g0, old)
            return out == merkle_ref(leaves, H), dict(n=n)
        return close_sized(run, d["nmin"], min_bound=d.get("min_bound", 0))
    raise ValueError(mode)


def judge_rpn(src):
    fn, g = oc.exact_version(src, "eval_rpn", {}); rng = random.Random(4)
    def gen(n):
        if n == 0: return ("v", rng.choice("abcd"))
        k = rng.randrange(n); return (rng.choice("+-*/"), gen(k), gen(n - 1 - k))
    def toks(t): return [t[1]] if t[0] == "v" else toks(t[1]) + toks(t[2]) + [t[0]]
    def ev(t, env):
        if t[0] == "v": return env[t[1]]
        a, b = ev(t[1], env), ev(t[2], env)
        return a + b if t[0] == "+" else a - b if t[0] == "-" else a * b if t[0] == "*" else a / b
    for n in range(0, 9):
        for _ in range(60):
            t = gen(n); env = {c: gf_rand(rng) for c in "abcd"}
            try: want = ev(t, env)
            except ZeroDivisionError: continue                  # 식 자체가 0으로 나눔(예: a/(b-b)): 표본에서 뺀다
            try: out = fn(toks(t), env)
            except Exception as e: return dict(verdict="반례", info=f"연산 {n}개: 실행 실패 {type(e).__name__}")
            if v(out) != v(want): return dict(verdict="반례", info=f"연산 {n}개 프로그램")
    return dict(verdict="표본 증명", info="프로그램 540개 각각은 모든 변수 값에서 증명, 프로그램 공간 전체는 아님")


def judge_ecmul(src):
    g = dict(PP=SECP_P); exec(compile(EC_SRC, "<ec>", "exec"), g); exec(compile(src, "<ecmul>", "exec"), g)
    fn = g["ec_mul"]; rng = random.Random(9)
    def ref_mul(k, P_):
        R = None
        for _ in range(k): R = ec_ref(R, P_)
        return R
    P0 = rand_point(rng)
    for k in range(0, 65):                                      # 작은 k 전수: 정의(반복 덧셈)와 비교
        try: out = fn(k, P0)
        except Exception as e: return dict(verdict="반례", info=f"k={k}: 실행 실패 {type(e).__name__}")
        if out != ref_mul(k, P0): return dict(verdict="반례", info=f"k={k}")
    try:
        if fn(SECP_N, SECP_G) is not None: return dict(verdict="반례", info="n·G != O")
        for _ in range(8):                                      # 큰 k: 준동형 (a+b)P = aP + bP
            a_, b_ = rng.randrange(SECP_N), rng.randrange(SECP_N)
            if fn(a_ + b_, SECP_G) != ec_ref(fn(a_, SECP_G), fn(b_, SECP_G)): return dict(verdict="반례", info="준동형 위반")
    except Exception as e:
        return dict(verdict="반례", info=f"실행 실패 {type(e).__name__}")
    return dict(verdict="유한 증명", info="k <= 64 전수, n·G = O, 큰 k 8쌍 준동형(모든 k는 아님)")


def baseline(key, src):
    d = D[key]
    if d.get("fspec") is None: return None
    env = d.get("env_float", d.get("env", {}))
    try:
        fn = oc.float_version(src, d["fname"], env)
        if key == "ecmul": return None
        a = d["fmake"](np.random.default_rng(3)); out = fn(*a)
        return not bool(d["fspec"](a, out))
    except Exception:
        return True


def one(args):
    key, kind, where, src = args
    import signal
    def _al(s, f): raise TimeoutError()
    signal.signal(signal.SIGALRM, _al); signal.alarm(25)
    try:
        r = judge(key, src)
    except TimeoutError:
        r = dict(verdict="시간 초과")
    except Exception as e:
        r = dict(verdict="반례", info=f"판정 중 실행 실패 {type(e).__name__}: {str(e)[:60]}")
    finally:
        signal.alarm(0)
```

```
    signal.alarm(8)
    try: b = baseline(key, src)
    except TimeoutError: b = True
    finally: signal.alarm(0)
    crash = False
    if kind != "원본":
        try:
            signal.alarm(5); crash = baseline_crash(key, src)
        except TimeoutError:
            crash = True
        finally: signal.alarm(0)
    return dict(key=key, kind=kind, where=where, src=src, B2=r.get("verdict"), info=str(r.get("info", r.get("bound", "")))[:150], BR=b, crash=crash)


def baseline_crash(key, src):
    """부동소수(또는 실제) 입력으로 돌리다 예외가 나면 '누구나 잡는' 변이체."""
    d = D[key]
    if d.get("fmake") is None: return False
    try:
        fn = oc.float_version(src, d["fname"], d.get("env_float", d.get("env", {}))); fn(*d["fmake"](np.random.default_rng(3))); return False
    except Exception:
        return True


def _work(args, q):
    import resource
    resource.setrlimit(resource.RLIMIT_AS, (1500 * 2 ** 20, 1500 * 2 ** 20))     # 1.5GB: 폭주는 MemoryError로
    try: q.put(one(args))
    except BaseException as e:
        q.put(dict(key=args[0], kind=args[1], where=args[2], src=args[3], B2="반례", info=f"판정 중 {type(e).__name__}", BR=True, crash=True))


def run_isolated(args, limit=40):
    """작업 하나를 자기 프로세스에서: 죽거나 멈춰도 전체가 서지 않는다."""
    q = mp.Queue(); p = mp.Process(target=_work, args=(args, q)); p.start(); p.join(limit)
    if p.is_alive(): p.kill(); p.join()
    try: return q.get(timeout=1)
    except Exception:
        return dict(key=args[0], kind=args[1], where=args[2], src=args[3], B2="반례", info="작업 프로세스 종료(메모리·시간)", BR=True, crash=True)


def main(keys=None, cap=150):
    keys = keys or list(D)
    jobs = []
    for k in keys:
        src = D[k]["src"]; jobs.append((k, "원본", "", src))
        ms = mutate.mutants(src); random.Random(0).shuffle(ms)
        for kind, where, line, s_ in ms[:cap]: jobs.append((k, kind, where, s_))
    done = set()
    if os.path.exists("domain_results.jsonl"):
        for l in open("domain_results.jsonl"):
            r = json.loads(l); done.add((r["key"], r["kind"], r["src"]))
    todo = [j for j in jobs if (j[0], j[1], j[3]) not in done]
    print(f"전체 {len(jobs)}, 남음 {len(todo)}", flush=True)
    t0 = time.time()
    from concurrent.futures import ThreadPoolExecutor
    with ThreadPoolExecutor(os.cpu_count()) as ex, open("domain_results.jsonl", "a") as fh:
        for i, r in enumerate(ex.map(run_isolated, todo)):
            fh.write(json.dumps(r, ensure_ascii=False) + "\n"); fh.flush()
            if i % 100 == 0: print(i, len(todo), f"{time.time() - t0:.0f}s", flush=True)
    out = [json.loads(l) for l in open("domain_results.jsonl")]
    keep = {(j[0], j[1], j[3]) for j in jobs}
    out = [r for r in out if (r["key"], r["kind"], r["src"]) in keep]
    json.dump(out, open("domain_results.json", "w"), ensure_ascii=False, indent=0)
    print("끝", f"{time.time() - t0:.0f}s", len(out))


if __name__ == "__main__":
    main(sys.argv[1:] or None)
```

stat_close.py — 21.4절: 확률 명세의 E 층과 S 층(큰수의 법칙·비율 편차·DKW)

```
"""stat_close: 확률 명세의 오라클 — 큰수의 법칙과 비율 편차(21.5절).

확률 명세는 "출력의 기댓값이 μ*이다"(몬테카를로 π: E[4·1{x²+y² ≤ 1}] = π, 허친슨: E[zᵀAz] = tr A) 또는
"출력의 분포가 F*이다"(난수: 균등분포)의 꼴이다.  값 하나로는 맞고 틀림이 정해지지 않으므로 두 층으로 나눈다.

  E 층(정확)   주어진 표본에서 코드의 값 = 의도한 식의 값.  대수 식이면 GF로 모든 표본값에서(20.2절),
               순서가 들어가면 유리수(순서가 있는 정확 산술)로 실제 표본에서.
  S 층(통계)   표본이 독립·동일 분포이고 X_i ∈ [a, b]이면, 어느 때나 유효한 비율 편차 판정:
                 N_k = N_0·2^k,  δ_k = δ / ((k+1)(k+2))  (Σ δ_k = δ),
                 |X̄_{N_k} - μ*| > (b - a) √(ln(2/δ_k) / (2 N_k))  이면 반례(유의수준 δ).
               (i) 명세가 참이면 끝까지 한 번이라도 반례를 낼 확률 <= δ   (호에프딩 + 합집합 상계)
               (ii) 기댓값이 μ ≠ μ*이면 강한 큰수의 법칙으로 X̄ -> μ, 문턱 -> 0 이므로 확률 1로 언젠가 반례.
                    대략 N >= 2 (b-a)² ln(2/δ_k) / (μ - μ*)² 에서 잡힌다.
               분포 명세는 DKW 부등식  P(sup|F_N - F*| > ε) <= 2 exp(-2Nε²)  를 같은 일정으로 쓴다.
"""
import math, random
from fractions import Fraction as Fr
import numpy as np
import oracle_close as oc
from oracle_close import GF, gf_rand, ValueBranch

DELTA = 1e-6


def thr(N, k, width, delta=DELTA):
    return width * math.sqrt(math.log(2 / (delta / ((k + 1) * (k + 2)))) / (2 * N))


def anytime_mean(stat, target, width, N0=1 << 10, K=10, delta=DELTA):
    """stat(N) -> 표본 평균(앞 N개).  반환 dict(verdict, N, est, thr)."""
    for k in range(K + 1):
        N = N0 << k; est = stat(N); t = thr(N, k, width, delta)
        if not np.isfinite(est) or abs(est - target) > t:
            return dict(verdict="반례", N=N, est=est, thr=t, k=k)
    return dict(verdict="δ-증명", N=N, est=est, thr=t, k=K)


def anytime_dkw(sample, N0=1 << 10, K=8, delta=DELTA):
    """sample(N) -> [0,1) 위 값 N개.  균등분포 명세의 DKW 판정."""
    for k in range(K + 1):
        N = N0 << k; x = np.sort(np.asarray(sample(N), float))
        if len(x) != N or not np.all(np.isfinite(x)): return dict(verdict="반례", N=N, D=float("nan"), thr=0)
        D = max(np.max(np.arange(1, N + 1) / N - x), np.max(x - np.arange(N) / N))
        eps = math.sqrt(math.log(2 / (delta / ((k + 1) * (k + 2)))) / (2 * N))
        if D > eps: return dict(verdict="반례", N=N, D=float(D), thr=eps, k=k)
    return dict(verdict="δ-증명", N=N, D=float(D), thr=eps, k=K)


# ---------------------------------------------------------------- 몬테카를로 π
def mc_layers(src):
    fn_f = oc.float_version(src, "mc_pi", {})
    # E 층: 유리수 표본(배정밀도 난수는 이분 유리수라 정확히 옮겨진다)에서 코드의 값 = 4·#{x²+y² <= 1}/N
    fn_q, g = oc.exact_version(src, "mc_pi", {})
    rng = np.random.default_rng(11); pts = rng.random((512, 2))
    Q = [(Fr(float(a)), Fr(float(b))) for a, b in pts]
    try:
        out = fn_q(Q); want = Fr(4 * sum(1 for a, b in Q if a * a + b * b <= 1), len(Q))
        E = "통과" if Fr(out) == want else f"반례: {float(out):.6f} != {float(want):.6f}"
    except Exception as e:
        E = f"반례: 실행 실패 {type(e).__name__}"
    # S 층: 독립 균등 표본, 어느 때나 유효한 비율 편차
    big = np.random.default_rng(7).random((1 << 20, 2))
    def stat(N):
        try: return float(fn_f(big[:N].tolist()))
        except Exception: return float("nan")
    S = anytime_mean(stat, math.pi, 4.0)
    return E, S


# ---------------------------------------------------------------- 허친슨 대각합 추정
HUTCH = '''
def hutch(A, zs):
    n = len(A)
    s = 0
    for z in zs:
```